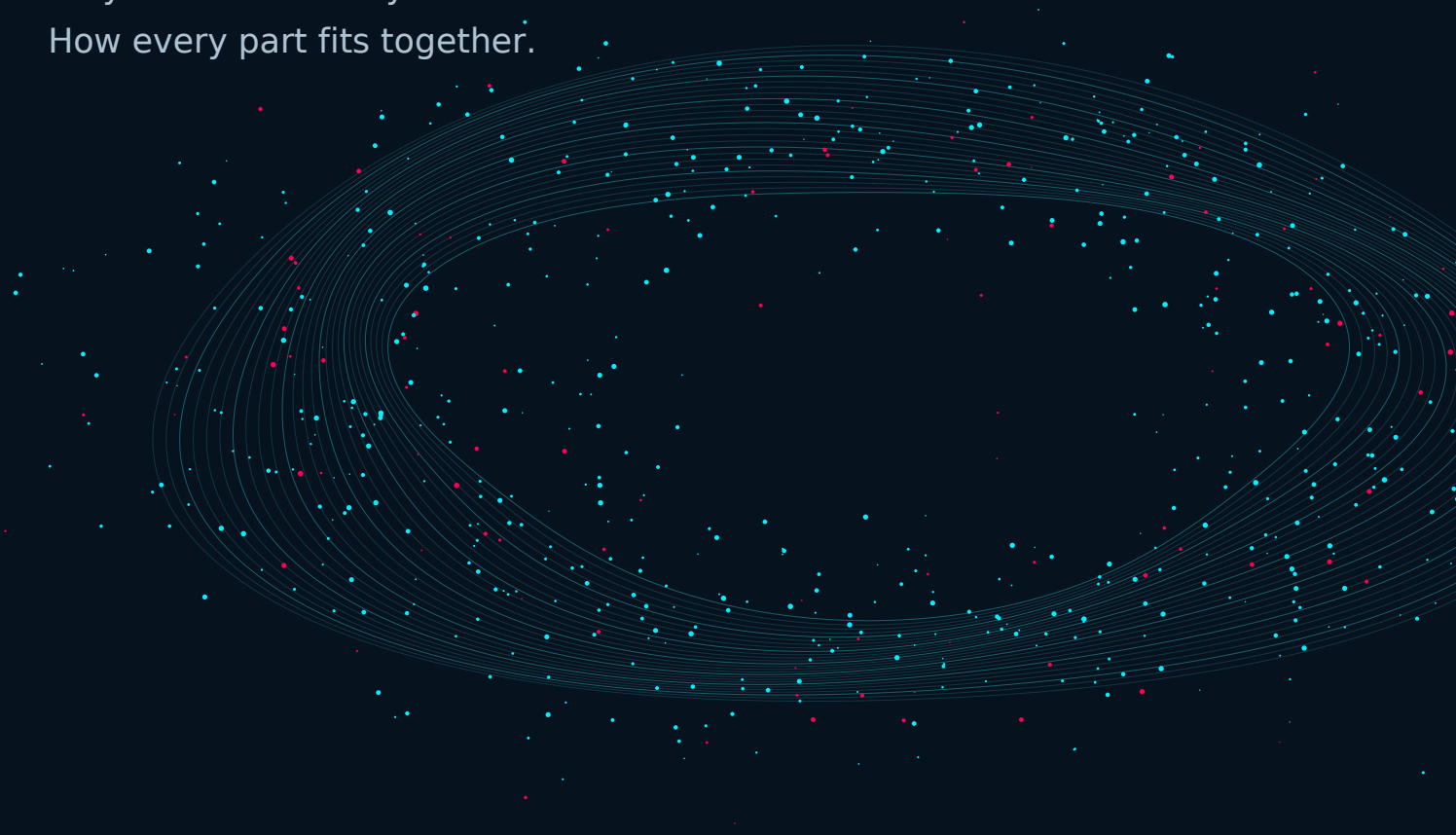
ALGORITHMIC SANCTUARY

RESONATE

The complete project walkthrough

Why it exists. What you built.
How every part fits together.



A detailed, source-grounded guide to the vision, visual engine, product demos, software architecture, and responsible next steps.

26 ROUTES

CUSTOM SHADERS

FULL SOURCE REFERENCE

READING MAP

Inside this walkthrough

Start with the story. Then follow the experience into the code.

01	The project in one clear explanation	4
02	Why I made it	5
03	Read the project honestly	6
04	The complete visitor journey	7
05	A tour of all 26 routes	8
06	The rest of the site map	9
07	Why the project looks this way	10
08	Everything I used—and why	11
09	The architecture at a glance	12
10	From HTML to an interactive page	13
11	How the cinematic entrance works	14
12	The neural hero, explained	15
13	The rest of the visual engine	16
14	Sound built in the browser	17
15	Echo Studio	18
16	Empathy Coach and Mood Genome	19
17	ResoQuests, Safe Circles, Heritage	20
18	The terminal: a project within the project	21

HOW TO USE IT / Chapters 01–08 for the big picture. Chapters 09–29 for implementation and honest technical defense. Chapters 30–33 for next steps and presentation practice. PDF bookmarks and contents entries are clickable.

READING MAP

Inside this walkthrough

Study the engineering, rehearse the presentation, and use the reference.

19 What happens when someone joins? 22

20 Domain design and shared state 23

21 Offline-first synchronization 24

22 The AI foundations, unpacked 25

23 Privacy and peer networking 26

24 Telemetry, memory, and the backend 27

25 How the monorepo holds together 28

26 What changed from Next.js 29

27 Run, build, and deploy it 30

28 What I verified 31

29 What I did not use—or finish 32

30 From prototype to responsible product 33

31 How to present RESONATE 34

32 Questions you should be able to answer 35

33 Glossary and final perspective 36

A Code-reading map, scene deep dives, and test plan 37

B Complete tracked-file catalog 40

C Every declared dependency 58

D Sources, reproducibility, and credits 67

HOW TO USE IT / Chapters 01–08 for the big picture. Chapters 09–29 for implementation and honest technical defense. Chapters 30–33 for next steps and presentation practice. PDF bookmarks and contents entries are clickable.

01 / START HERE

The project in one clear explanation

RESONATE is a cinematic, dark-themed portfolio and interactive product prototype for an emotional-wellbeing platform aimed primarily at Indian students aged 13–18. Its identity is “Algorithmic Sanctuary”: a proposed alternative to attention-driven social platforms, built around being heard, understanding emotions, and connecting with a small number of supportive peers.

Two projects inside one repository

The **present-day website** communicates this idea through 26 explicit routes, custom WebGL visuals, sound synthesis, animated storytelling, a simulated journal, a scripted chat, and an unusually substantial browser terminal. The **proposed wellbeing platform** adds local speech transcription, trustworthy emotional analysis, private peer circles, longitudinal mood tracking, and school-supported safeguarding. These are not all implemented end to end.

A presentation-ready introduction

“I built RESONATE to explore a different role for technology in student life. Instead of encouraging comparison and endless scrolling, the concept prioritizes emotional expression, useful reflection, and meaningful peer support. This repository is the interactive portfolio and prototype that makes that vision tangible. It combines a complete visual identity with frontend engineering and early foundations for local-first, privacy-conscious systems.”

What is demonstrably substantial

The site is not just a static landing page. It has a shared application shell, grouped navigation, reusable motion components, custom GLSL shaders, GPU-rendered particle scenes, browser-generated audio, stateful demos, domain classes, concurrency-related code, and an API workspace. Its strongest completed achievement is the frontend experience and technical communication.

What this document will not pretend

A fixed chat response is not a deployed LLM. A graphical heatmap is not collected clinical data. AES-GCM helper methods alone do not make an application end-to-end encrypted. A successful Vite bundle is not a successful TypeScript build or a safety certification. Being precise about these differences makes the project easier—not harder—to defend.

Evidence: `replit.md`; `artifacts/resonate/src/App.tsx`; `artifacts/resonate/src/app/about/page.tsx`; feature pages; infrastructure modules.

02 / MOTIVATION AND INTENDED IMPACT

Why I made it

The problem described by the project

The About and Manifesto pages focus on loneliness, exam pressure, cultural expectations, and performative online connection. They explicitly reference JEE, NEET, Board examinations, family dynamics, and the difficulty of expressing emotional vulnerability. The target is therefore not an abstract “everyone”; it is a recognizable Indian student context.

The original ideas file uses deliberately confrontational language about attention platforms. The useful underlying design thesis is that a digital experience can optimize for understanding and constructive action instead of popularity, comparisons, or time spent scrolling. Those views are the project's framing, not independently established findings about any particular platform.

Suggested first-person explanation

“I wanted to build something that felt relevant to students around me. Academic performance gets measured constantly, but emotional wellbeing can remain invisible. RESONATE asks what would happen if the same engineering effort used to maximize engagement were redirected toward helping a student express a difficult feeling, reframe an overwhelming thought, or find a genuinely supportive small group.”

Turning motivation into product choices

Voice-first expression lowers the effort of starting a journal when typing feels burdensome. **Culturally specific coaching** makes examples more relevant than generic advice. **Small circles** are intended to privilege depth over audience size. **Mood patterns** make reflection longitudinal rather than restricted to one conversation. **Micro-challenges** turn passive reflection into a small next action. **Family storytelling** extends connection across generations.

Privacy follows from the sensitivity of the subject matter: a journal can reveal much more than an ordinary social post. Local processing is therefore an architectural intention, not just a performance slogan. But local processing must be combined with secure storage, trustworthy code, clear consent, and tested data handling to deliver that intention.

What success should eventually mean

For the prototype: visitors understand the problem, remember the identity, and can follow the demo. For a future pilot: students understand consent, feel safe, can complete tasks accessibly, and receive appropriate human support when necessary. Health outcomes need ethical evaluation and credible evidence; they cannot be inferred from visual polish or simulated metrics.

Evidence: `.migration-backup/IDEAS.md`; `src/app/about/page.tsx`; `src/app/manifesto/page.tsx`; six feature pages. `src/` paths hereafter refer to `artifacts/resonate/src/` unless otherwise stated.

03 / IMPLEMENTATION MATURITY

Read the project honestly

Four levels of evidence

Working interface: the code contains real event handling and state transitions that support an interaction. This does not by itself prove that every device or route has been browser-tested.

Scripted demonstration: the interface works, but the “intelligence,” statistics, or output is predetermined or randomized. It is a prototype of an experience, not a validated service.

Engineering foundation: meaningful code exists, but is not connected to the live feature flow or lacks infrastructure needed to operate end to end.

Product vision: explanatory copy, roadmap items, or promises describe what the future product should do.

Classification at this snapshot

- **Working interfaces:** route switching, navigation state, cinematic entry sequence, magnetic hover effects, synthesized UI sounds, terminal parsing, and browser-local waitlist storage.
- **Scripted demos:** Echo Studio transcription and reframe; Empathy Coach responses; Mood Genome heatmap and insight; most terminal network/security outputs.
- **Foundations:** WebGPU scalar compute pipeline, lexical-analysis worker, vector clocks/LWW map, AES-GCM helper, WebRTC mesh class, domain entities, event bus, and telemetry buffer.
- **Vision or unverified claims:** deployed Whisper, fine-tuned CBT/DBT LLMs, reinforcement-learning personalization, live counselor oversight, automatic guardian intervention, outcome percentages, node counts, and latency guarantees.

Why this matters during evaluation

A reviewer may ask “Does this actually use AI?” The strongest answer is specific: “The current chat and voice demo are scripted. I also wrote a lexical worker and a WebGPU compute primitive, but those do not constitute a trained language model. The next implementation phase connects an evaluated model to the interface with proper safety controls.”

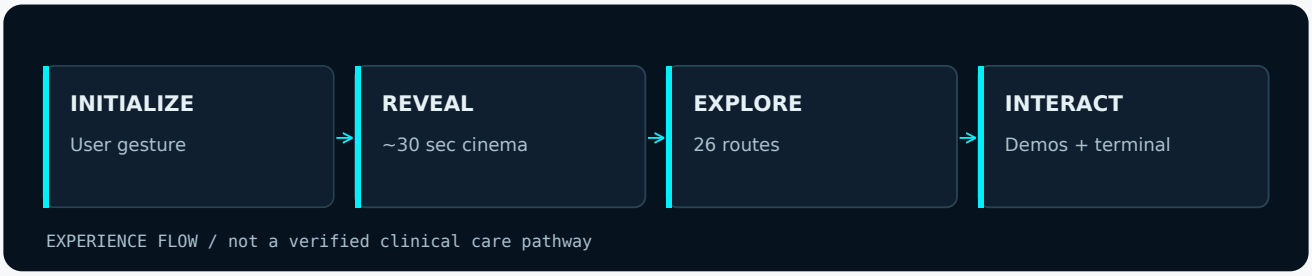
A second common question is “Is it private?” A careful answer is: “The design prioritizes local processing and contains browser cryptography primitives, but the full data lifecycle and security model are not finished or audited.”

Important: this walkthrough describes source inspection and the checks recorded later. It does not certify clinical effectiveness, legal compliance, live deployment, or universal browser compatibility.

Evidence: feature handlers; `presentation/components/chat/AdvancedChat.tsx`; `infrastructure/`; `workers/`; terminal waitlist flow.

04 / FROM ARRIVAL TO EXPLORATION

The complete visitor journey



1. Enter the world

`main.tsx` mounts the React application. The global shell adds grain, a scanline, a custom cursor, and a custom context menu. Inside the providers, the cinematic loader initially prevents the actual page children from mounting. This is a deliberate reveal rather than a conventional data-loading indicator.

2. Initialize sound and the cinematic sequence

The user clicks “Initialize Sequence.” The acoustic engine creates its browser audio graph. Timed terminal logs and large animated statements play over a particle canvas. After about 28.5 seconds, the final transition begins; its 1.5-second animation brings the total to roughly 30 seconds before the main content appears.

3. Understand the proposition

Home pairs a neural-particle hero with “old web” positioning, headline statistics, and calls to explore the manifesto, engine, or terminal. The live-looking memory number is randomized display data; it does not measure actual memory allocation.

4. Explore the product and its explanation

Feature pages show the proposed student experience. Engine, Technology, CRDT, and Telemetry tell the technical story. About, Founders, and Manifesto provide identity. Impact, Schools, and Roadmap pitch adoption. Protocol, Privacy, and Terms communicate intended rules and boundaries.

5. Try an interaction

A visitor can trigger a scripted transcription, send a message to the scripted chat, or explore the terminal. The terminal's `join` flow actually stores a browser-local record; the separate Join form has no implemented submission handler.

The experience trade-off

The cinematic opening is memorable, but a required 30-second sequence on each fresh app mount can obstruct a stressed student, a repeat visitor, or a reviewer. A production version should offer skip, mute, reduced-motion, and session-level “already seen” behavior. Atmosphere should introduce the product, not delay access to help.

Evidence: `main.tsx`; `App.tsx`; `presentation/layouts/AppProvider.tsx`; `presentation/components/animations/CinematicLoader.tsx`; Home and Terminal.

05 / NARRATIVE AND PRODUCT

A tour of all 26 routes

Main identity and conversion

• `/` **— Home:** neural particle background, mission-led hero, manifesto section, statistics, engine teaser, terminal call to action. It establishes identity before asking for action.

• `/about` **— About:** student-focused origin story, mission, values, and a written timeline. A neural canvas visually ties it back to Home.

• `/manifesto` **— Manifesto:** opinionated, animated text explaining the rejection of superficial engagement. It is positioning, not a research paper.

• `/founders` **— Founders:** animated profiles for Anuj Phulera and Aarav Choudhary using the two attached anime images as stylized identities—not literal founder portraits.

• `/join` **— Join:** early-access form for name, email, age, and school. HTML validation exists; an API-backed registration flow does not.

• `/terminal` **— Terminal:** command-driven product exploration and local waitlist interaction; the most elaborate interactive page in the repository.

Feature family

• `/features` **— Feature overview:** the entry point to the six product ideas, with branded motion and navigation to detail pages.

• `/features/echo-studio`**:** demonstrates voice-journal transcription and reframing with a hard-coded example and timers.

• `/features/empathy-coach`**:** hosts `AdvancedChat`; user messages are real local state, but the assistant reply is fixed.

• `/features/mood-genome`**:** demonstrates a mood-pattern dashboard using a randomized 35-cell heatmap and a written sample insight.

• `/features/resoquests`**:** pitches emotional micro-challenges. “Accept Quest” currently plays a sound; it does not assign or persist a task.

• `/features/safe-circles`**:** explains small supportive groups, encryption, moderation, and matching; it does not connect actual peers.

• `/features/heritage-echoes`**:** presents intergenerational storytelling and regional-language transcription/translation; no upload or translation flow is implemented.

How to explain the information architecture

The site moves from **why** to **what**, then **how**, then **who**, then **how to participate**. Separate feature pages give each concept room to breathe and allow reusable visual motifs to support the topic. The cost is repeated layout code and more places for product claims to drift out of sync.

Evidence: explicit `Route` declarations in `App.tsx`; corresponding `src/app/**/page.tsx` files. The fallback 404 is additional and is not counted among the 26 explicit routes.

06 / ENGINEERING, TRUST, AND ECOSYSTEM

The rest of the site map

Technical explanation

• `/engine`: high-level architecture panels over a mesh-topology canvas. It discusses local GPU analysis, peer connectivity, and related foundations, sometimes more strongly than the implementation supports.

• `/technology`: a second technical pitch covering edge rendering, WebGPU, P2P, and CRDTs. Its “Next.js 15” description is stale: the current frontend is Vite/React, and the preserved package declares Next.js 16.2.9.

• `/crdt`: an explanatory page for offline-first synchronization. The vector-clock code is real, but a deployed journal sync service is not demonstrated.

• `/telemetry`: styled operational metrics. Counts, uptime, latency, and interventions are literal display values, not live observability results.

Ecosystem and adoption

• `/impact`: globe-backed storytelling with “10K+,” “85%,” “4.8,” and “1.2M” display metrics. No evaluation dataset in the repository substantiates these outcomes.

• `/schools`: presents the school/counselor proposition and links toward Join. It does not provide a functioning counselor dashboard.

• `/roadmap`: scroll-linked 3D timeline with four planned phases. Status labels are hard-coded content, not evidence of completed deployments.

• `/careers`: renders role cards with application links to the terminal rather than a recruitment backend.

• `/blog`: presents four article-summary records. There is no separate article-detail route in the application switch.

• `/archives`: another heritage/archive-themed narrative using the DNA visual motif; not a persistent document archive.

Rules and reassurance

• `/protocol`: states intended no-harvesting, anonymity, moderation, and emergency principles.

• `/privacy`: describes zero-knowledge intentions, telemetry, and SOS overrides. Policy copy must be reconciled with real data flows before launch.

• `/terms`: includes behavior rules and a medical disclaimer. It is not a substitute for product, safeguarding, and legal review.

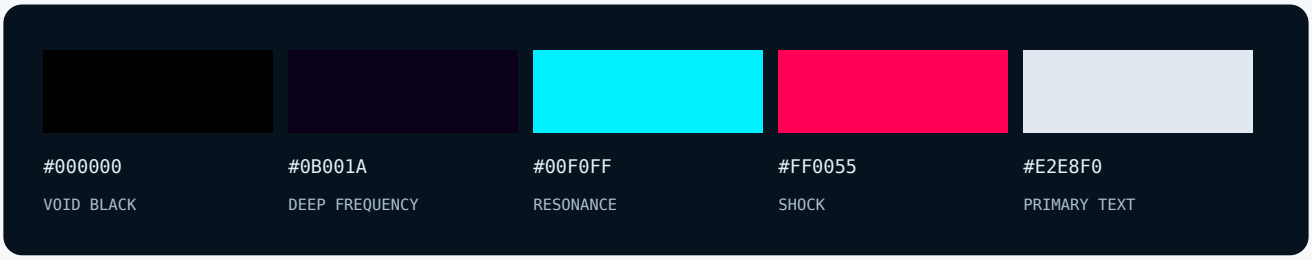
Fallback behavior

`src/app/not-found.tsx` is the active fallback imported by `App.tsx`. `src/pages/not-found.tsx` also exists, but is not the fallback selected by the current router. The directory name `app` is an organizational remnant, not evidence that Next.js routing remains active.

Evidence: `App.tsx`; technical, ecosystem, trust, and fallback page sources.

07 / VISUAL IDENTITY AND DESIGN LANGUAGE

Why the project looks this way



A sanctuary expressed as a system

RESONATE's visual identity is deliberately closer to a cinematic operating system than to a conventional wellness brochure. Absolute black gives the particle light room to read. Deep purple creates depth. Cyan signals connection and operational activity; magenta signals critique, danger, or interruption. Off-white carries primary text while slate is used for secondary copy.

Three typographic voices

Oswald is the display voice: condensed, oversized, uppercase headings with tight tracking. **Inter** provides the sans-serif body foundation. **Geist Mono** gives logs, system labels, statistics, and captions their terminal character. The Vite HTML file loads them with Google Fonts links; CSS variables assign their roles.

The interface vocabulary

Thin borders, crosshair corners, coordinate readouts, grid backgrounds, frosted dark panels, outline text, glowing edges, and uppercase labels form a consistent visual vocabulary. Noise and scanlines stop flat surfaces from feeling empty. Mouse-reactive particles and magnetic links make the system feel physically responsive.

Why these choices help a portfolio

A portfolio must make the underlying idea memorable. Here, visual effects are not isolated decorations: mesh scenes suggest connection, a DNA helix suggests inheritance, and a journey canvas suggests progression. The system metaphor provides coherence across a large number of pages.

Where the design needs restraint

The emotional-support audience may not always benefit from aggressive language, warning colors, simulated threat elimination, flashing text, or loud sound. Slate `#475569` against black also deserves a proper contrast audit for body text. A real student-facing version should balance identity with calm, readable, predictable interaction.

Do not describe the visuals as proof of privacy or AI. A terminal-style “encrypted” label and a glowing status indicator are communication choices; actual guarantees must come from implementation and verification.

Evidence: `src/index.css`; `artifacts/resonate/index.html`; `.migration-backup/IDEAS.md`; shared presentation components.

08 / THE ACTIVE TECHNOLOGY STACK

Everything I used—and why

The application foundation

React 19 structures the site into composable pages, shared layouts, and reusable interactions. State and effects drive the terminal, transcript, loading sequence, and chat. **TypeScript 5.9** adds contracts for props, events, domain values, and API responses. It catches errors that a Vite transpile-only build can miss.

Vite 7 is the current dev server and frontend bundler. It provides a React-oriented development loop and builds a static client application. **Wouter 3** handles client-side routing; it replaces the original Next.js file-based route mechanism. **pnpm workspaces** organize the frontend, API, shared libraries, and tooling under one lockfile.

Visual and interaction technologies

Tailwind CSS 4 supplies utility classes and theme tokens, compiled through `@tailwindcss/vite`. **Three.js** provides rendering objects and shader integration. **React Three Fiber** expresses Three scenes as React components. **Drei** supplies helper abstractions used by some canvas components.

GSAP + ScrollTrigger handle imperative scroll reveals and coordinated transitions. **Framer Motion** handles declarative entrances, hover motion, animated presence, and scroll-linked values. **Anime.js 4** handles the loader's final transition. **Lenis** changes the feel of wheel scrolling. **Lucide React** supplies the navigation and interface icons.

Data and support technologies

Zustand defines a small empathy store, though it is not a complete persisted product state layer. **TanStack React Query** is mounted globally and also powers generated API hooks; mounting the provider does not mean feature data comes from a server. **Express 5, Pino, CORS, Zod, OpenAPI, and Orval** form an API scaffold around a health endpoint. **Drizzle + node-postgres** provide database plumbing, but the schema is currently empty.

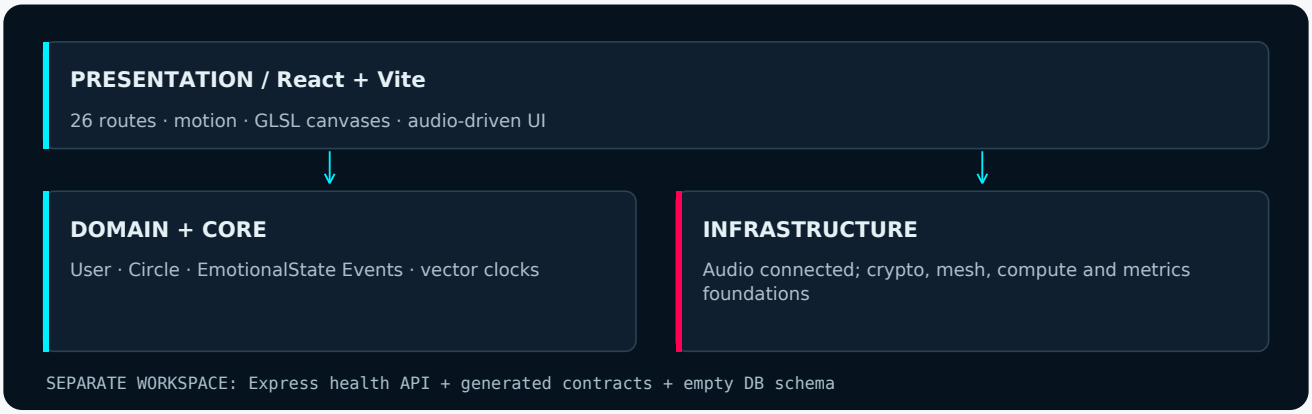
Native browser APIs matter too

Web Audio synthesizes the UI sounds. Web Crypto implements the AES-GCM/PBKDF2 helper. Web Workers contain a lexical-analysis demo. WebGPU contains an experimental compute primitive. WebRTC contains an unfinished peer mesh. `localStorage` is used by the terminal waitlist.

Evidence: workspace and package manifests; `App.tsx`; Vite config; `lib/`; native API modules. The dependency appendix distinguishes declared tools from demonstrated feature use.

09 / REPOSITORY AND DEPENDENCY BOUNDARIES

The architecture at a glance



Presentation: what the visitor sees

`artifacts/resonate/src/app/` holds page components. `presentation/` holds canvas scenes, shader programs, animation helpers, global layouts, navigation, chat UI, and the empathy store. `components/ui/` contains a large Radix/shadcn-style component library; its presence does not mean every component appears on the product pages.

Domain: what the product means

`domain/entities/` models a User and a ResonanceCircle. `domain/value-objects/EmotionalState.ts` models emotional metrics and thresholds. These classes express business concepts without JSX. Their isolation makes them easier to test, but they are not proof that the live website instantiates or persists these objects.

Core and infrastructure: how foundations could work

`core/` contains identity/event primitives, an event bus, vector clocks with an LWW map, and a resource-tracking manager. `infrastructure/` contains adapters or experiments for audio, cryptography, GPU compute, telemetry, and peer networking. The audio service is visibly integrated; many other foundations are not connected to active feature pages.

Workspace services

`artifacts/api-server/` is an Express service. `lib/api-spec/` defines its contract; generated React and Zod packages consume that contract. `lib/db/` is PostgreSQL/Drizzle scaffolding. `artifacts/mockup-sandbox/` is a separate component-preview environment, not another RESONATE product route.

The architectural lesson

The folder layout resembles layered or domain-oriented design, but directories alone do not establish Clean Architecture or full CQRS. A future application-service layer should orchestrate consent, input, analysis, storage, and events. Right now, the safest description is “a presentation-heavy prototype with separated domain and infrastructure foundations.”

Evidence: repository directory tree; `App.tsx`; `core/`; `domain/`; `infrastructure/`; API and shared-library entry points.

10 / STARTUP, PROVIDERS, AND ROUTING

From HTML to an interactive page

The boot chain

`index.html` defines metadata, font links, the root element, and the `src/main.tsx` module. `main.tsx` imports global CSS and calls `createRoot(...).render(<App />)`. There is no server-rendered page pipeline in the current application.

Provider nesting is part of the design

A single `QueryClient` is created outside the component. `QueryClientProvider` wraps Wouter. The router base is derived from Vite's `BASE_URL`, with its trailing slash removed. Global visual layers and cursor/context-menu components sit outside the content providers; Lenis then wraps AppProvider, which manages the cinematic gate. GlobalNav and the routed main element mount only after that gate finishes.

Why explicit routes matter

`App.tsx` imports every page eagerly and selects one through `Switch` and `Route`. The familiar `src/app/.../page.tsx` paths are not automatically discovered. To add a route, a developer must create the component, import it, and register its path. A final pathless route catches unknown locations.

Navigation behavior

GlobalNav groups eight primary items with submenus for features, engineering, identity, impact, and policies. Wouter's current location drives the active state, including recognition of child links. Framer Motion animates hover panels and the active dot; AcousticResonator provides hover and click cues.

Engineering consequences

Eager page imports contribute to the large initial bundle. Route-level `lazy()` imports and `Suspense` would allow expensive pages and their canvases to load on demand. Hover-only icon menus need accessible names, keyboard behavior, and touch alternatives. Context-menu actions use `window.location.href`, causing full navigation and ignoring Wouter's base-path abstraction; they should eventually use base-aware routing.

The site-level metadata is useful, but a client-rendered SPA does not automatically provide independently rendered route metadata or the SSR benefits claimed by older copy.

Evidence: `index.html`; `main.tsx`; `App.tsx`; `presentation/components/navigation/GlobalNav.tsx`; `presentation/components/ui/ContextMenu.tsx`.

11 / TIMERS, SOUND, AND REVEAL

How the cinematic entrance works

What the loader really does

`AppProvider` starts with `loading = true`. `CinematicLoader` initially displays a click-to-initialize layer. Clicking changes local interaction state and calls the audio singleton's asynchronous `initialize()` method. Starting sound from a user gesture is important because browsers restrict autoplay.

The loader schedules a series of log messages from 2 to 26 seconds. Error messages trigger a deep thud; other messages trigger a hover ping. Separate text-sequence records define larger statements with start/end windows. `CinematicLoaderCanvas` supplies the GPU-rendered atmosphere.

The final transition

At 28.5 seconds the code plays a riser, then calls Anime.js `animate` on the loader container. It fades opacity, scales the element, and blurs the screen over 1,500 milliseconds using `ease: "inExpo"`. Its `onComplete` callback releases the gate in AppProvider. Framer Motion's `AnimatePresence` also participates in exit handling.

Why it was built this way

The opening makes the visitor feel they are booting into a different digital environment. Sound, logs, particles, and typography communicate a single idea: leaving the old attention economy and entering RESONATE. It is experience design expressed through coordinated browser APIs.

Important code-level limitations

The timeout cleanup returned by `startSequence()` is returned from an event handler, not a React effect. React will not automatically invoke it on unmount. Timers should instead be registered in a ref or managed by an effect with a real cleanup function. Audio initialization failures should have a silent fallback; the reveal should not depend on successful sound initialization.

The loader's `loading` state is in component memory. A new app mount resets it; no session persistence is implemented here. It also withholds child routes rather than letting meaningful content render underneath for immediate access.

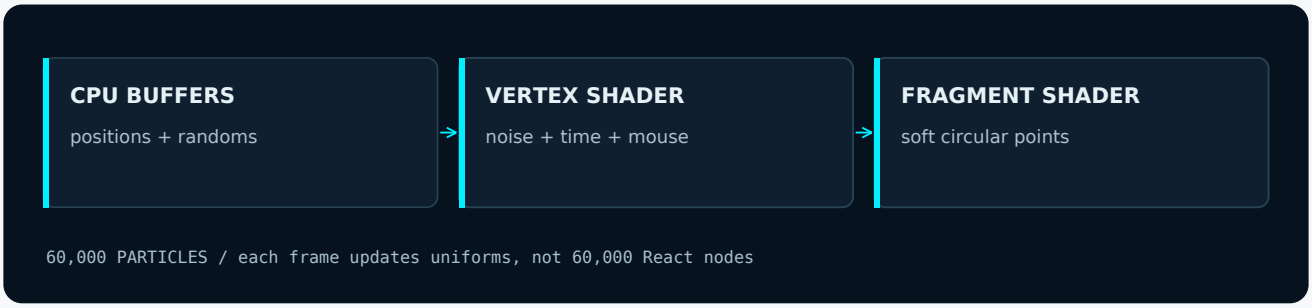
A better next iteration

Offer a semantic keyboard-operable Start button, a prominent Skip option, mute/volume settings, and reduced-motion behavior. Make the cinematic sequence optional after the first visit. Distinguish theatrical progress from actual resource-loading progress, and always allow direct access to support information.

Evidence: `presentation/layouts/AppProvider.tsx`; `presentation/components/animations/CinematicLoader.tsx`; `infrastructure/audio/AcousticResonator.ts`.

12 / REAL GPU GRAPHICS—NOT A VIDEO

The neural hero, explained



Geometry and data

`NeuralNetworkCanvas.tsx` creates **60,000 particles**. Two `Float32Array` buffers hold positions and random attributes, with three values per particle. Positions are spread across a wide cinematic volume: roughly 40 units horizontally, 20 vertically, and 10 in depth. `useMemo` avoids regenerating those buffers on every React render.

Shader-driven motion

The custom vertex shader receives position, random attributes, elapsed time, and mouse coordinates. Its simplex-noise/curl-noise functions create flowing motion. The fragment shader discards pixels outside the circular point footprint and softens the remaining particle using a distance-based gradient. Additive blending gives overlapping points their luminous character; disabling depth writes helps transparent particles combine visually.

Mouse interaction

A window mouse listener converts screen coordinates into normalized values and scales them using the viewport. `useFrame` updates the time uniform and smoothly interpolates the current mouse vector toward the target with `lerp(..., 0.05)`. This avoids abrupt jumps while keeping the particle field reactive.

Why use shaders here?

React does not update thousands of particle components every frame. Instead, typed buffers describe geometry and the GPU applies the shader program across particles in parallel. The JavaScript render loop updates a few uniforms while the graphics hardware does the repeated visual work.

The Vite bridge

An inline `glsl-loader` plugin matches `.glsl`, `.vert`, and `.frag` imports and returns a JavaScript module exporting the shader source as a string. Three's ShaderMaterial then receives that string. GLSL for visual rendering and WGSL for WebGPU compute are separate technologies with separate roles in this repository.

Performance caveat

A 60,000-point scene is real work. It needs measurement on low-end mobile hardware, a static/WebGL-unavailable fallback, and quality scaling. The resolution uniform is initialized from the window and is not updated by a dedicated resize handler in this component. No universal frame-rate guarantee follows from the particle count alone.

Evidence: `presentation/canvas/NeuralNetworkCanvas.tsx`; `presentation/shaders/neural.vert`; `neural.frag`; `vite.config.ts`.

13 / SCENES, MOTION, AND CLEANUP

The rest of the visual engine

A library of visual metaphors

The presentation canvas directory includes a cinematic loader scene, a mesh topology, the neural field, a singularity swarm, a DNA helix, an audio visualizer, fluid motion, an impact globe, a prism, a roadmap scene, and a vector field. Each source file represents a reusable scene rather than a route by itself. Not every scene is necessarily imported into the active route graph.

The singularity implementation allocates **150,000 particles** and contains postprocessing imports for bloom, chromatic aberration, and noise. That is a particularly demanding effect, and its presence should not be confused with every page rendering that many particles. The current typecheck also reports an unresolved direct `postprocessing` import there.

Different animation tools do different jobs

GSAP is useful for imperative control of DOM elements and ScrollTrigger timelines. The Engine page uses `gsap.context` and returns `ctx.revert()` to undo animations on unmount. **Framer Motion** is useful when animation follows React state: chat messages appear, dropdowns enter and leave, and active navigation elements move. The Roadmap combines scroll progress with a 3D camera-oriented scene.

Anime.js runs the loader transition. **Lenis** feeds a smooth-scroll engine through `requestAnimationFrame`. These tools can coexist, but they should not compete to animate the same transform or opacity without explicit ownership.

Reusable interaction components

`MagneticElement` measures the pointer relative to its element center, multiplies that offset by a configurable strength, and lets a spring animate the wrapper. `CustomCursor` follows the pointer and grows when the target is interactive; it returns nothing on narrower viewports. `TextReveal`, `ScrambleText`, `MatrixRain`, and transition helpers build the system aesthetic.

Lifecycle improvements worth making

Lenis is destroyed during cleanup, but the recursively scheduled animation-frame callback is not explicitly canceled. The global GSAP orchestrator kills all ScrollTriggers on cleanup rather than just its own. Home also lacks the same scoped GSAP cleanup pattern used by Engine. Resource lifecycles should be scoped so changing routes does not leave old work running or destroy another component's animations.

Evidence: `presentation/canvas/`; `presentation/components/`; `presentation/hooks/useGSAPOrchestrator.ts`; `presentation/layouts/LenisProvider.tsx`; Home and Engine effects.

14 / ACOUSTICRESONATOR

Sound built in the browser

No prerecorded soundtrack is required

`AcousticResonator` is a singleton service built with the Web Audio API. Its private constructor and `getInstance()` method keep callers on one shared instance. `initialize()` guards repeated setup and creates an AudioContext, master GainNode, ConvolverNode, and lowshelf BiquadFilterNode.

The audio graph

The lowshelf filter boosts bass around 300 Hz. Its output goes both directly to the master gain and through the convolver, creating dry and reverberated sound paths. The master gain connects to the destination. The impulse response is generated procedurally from random noise with a decaying envelope in a stereo AudioBuffer.

Small events become audible feedback

A hover ping uses a sine oscillator that sweeps from about 1,200 to 2,000 Hz with a brief gain envelope. A click thud uses a triangle oscillator that falls from about 150 to 20 Hz with a stronger, longer decay. Other methods provide data ticks and cinematic emphasis. Navigation, magnetic wrappers, the loader, terminal, and demos call these methods.

Why this is a good engineering detail

Procedural sound keeps the system identity consistent without requiring a collection of media files. An oscillator can be parameterized for duration, frequency, envelope, and gain. A singleton avoids creating a full AudioContext for every hover. Guard clauses allow calls to return before initialization rather than crashing.

Why a production version needs controls

The master gain is set to 0.8 and the filter applies a bass boost, so effects are not designed as an inherently silent or subtle accessibility layer. Add mute, volume, session preferences, and audio-context resume handling. Audio errors should not block navigation or essential content. Hover feedback should be rate-limited to avoid excessive repeated sounds.

The sounds are real generated UI audio. They are not speech recognition, sentiment detection, or proof that a microphone is recording. In particular, Echo Studio's “Initialize Microphone” button currently triggers a simulation rather than microphone capture.

Evidence: `infrastructure/audio/AcousticResonator.ts`; loader; GlobalNav; MagneticElement; Terminal and Echo Studio event handlers.

15 / A CLEAR EXAMPLE OF THE PROTOTYPE BOUNDARY

Echo Studio

The product idea

Echo Studio is intended as a private voice journal. A student speaks a difficult thought, sees a transcript, receives an explanation of a possible cognitive distortion, and gets a more balanced reframe. The example in the code focuses on catastrophic thinking about an exam.

What the current interaction actually does

The button invokes `simulateTranscription()`. It exits if a recording simulation is already active, plays a sound, sets `recording` to true, and resets transcript/analysis state. The full transcript is a literal string: “I feel like I'm going to fail this exam and my entire life is completely ruined.”

A 50-millisecond interval appends characters to the transcript. At the end, it clears the interval, stops the recording state, plays a tick, and waits another 800 milliseconds before setting the analysis object. That object always labels “Catastrophizing / Fortune Telling” and supplies the same alternative thought about one test not determining an entire future.

What is real and what is simulated

The stateful reveal, button disabling, typewriter display, and animated result panels are genuine frontend work. The audio input, Whisper transcription, emotion understanding, and reframe generation are not performed. There is no MediaRecorder or microphone acquisition in this page's handler and no model loading or inference request.

How to explain it well

“I designed and implemented the voice-journal experience, including the transcript stream and analysis presentation. For this version, I used a deterministic sample so I could focus on the interaction. The next milestone is actual consent-based microphone capture and evaluated local transcription—not relabeling the existing timer as AI.”

A realistic implementation plan

Add clear consent and recording state; capture audio; handle permission denial; choose an on-device model appropriate to the hardware; load it with progress and cancellation; show an editable transcript; and only then run an evaluated reflection workflow. Include deletion and encryption rules before storing sensitive content. Do not promise “instant” or “zero latency” without measurements.

Timer cleanup also belongs in the next pass: navigating away should not leave intervals or delayed analysis callbacks active.

Evidence: `src/app/features/echo-studio/page.tsx`, especially `simulateTranscription` and the hard-coded `analysis` object.

16 / DIALOGUE AND EMOTIONAL PATTERNS

Empathy Coach and Mood Genome

Empathy Coach: interface vs intelligence

The feature page embeds `AdvancedChat`. That component maintains a message list, textarea state, and a typing flag. Send rejects empty input or concurrent sending, appends the user's message, clears the field, and uses two timers to simulate processing and typing. After roughly three seconds, it returns the same Board-exam/DBT-themed response regardless of the message.

The animated typing dots, message entry transitions, keyboard handling, disabled button, and automatic scrolling are real UI behaviors. The claimed fine-tuned LLM, CBT/DBT specialization, live heart-rate interpretation, self-harm detection, and guardian/helpline override are not implemented in that chat component. The “E2E Encrypted” label is not backed by a call to CipherProtocol in its send path.

Mood Genome: a dashboard concept

Mood Genome's goal is to show patterns over time, possible triggers, and useful interventions before a difficult day. The current page creates 35 heatmap cells using `Math.random()` during render. Its Sunday-night “40% higher” insight is written sample copy. There are no dated mood records, analytics queries, prediction model, or underlying evidence for that insight.

Why both prototypes are still useful

The chat explores how to make a reflection space readable and responsive. The heatmap explores a compact visual language for longitudinal emotion. These are valid experience prototypes; the correct next step is to replace sample outputs with trustworthy data sources, not to claim that visual plausibility is clinical validity.

What should come next

For chat: response provenance, model limits, evaluated safety behavior, session boundaries, deletion, and human escalation design. The initial message should not claim to know physiological data the system never collected. For the dashboard: an explicit mood-entry model, timestamps, timezone handling, missing-data states, stable rendering, transparent calculations, and uncertainty explanations.

A student must be able to tell the difference between a self-reported score, a model estimate, and a professional judgment. None of these should be presented as interchangeable, and the prototype must not be used as an emergency-care system.

Evidence: `presentation/components/chat/AdvancedChat.tsx`; Empathy Coach and Mood Genome pages; `presentation/store/empathyStore.ts`.

17 / THE WIDER PRODUCT VISION

ResoQuests, Safe Circles, Heritage

ResoQuests: make support actionable

The page presents short emotional micro-challenges, such as grounding exercises or a brief voice note. It proposes personalization from mood patterns and school context. Its current `Accept Quest` handler plays a sound only: no task state, progress, reward, schedule integration, or reinforcement-learning policy is applied.

A functional next version needs a small reviewed task catalog, clear completion and dismissal controls, optional reminders, local persistence, and a way to evaluate whether an intervention is appropriate. It should never claim to measure cortisol from a simple text or browser interaction.

Safe Circles: connection without a public audience

The page pitches intimate peer communities with shared challenges, encryption, moderation, and aggregate oversight. Its capacity card says **4–8** participants, while `ResonanceCircle.create()` rejects a capacity above **6**. That inconsistency should be resolved before turning copy into requirements.

The mesh background is visual storytelling, not active network membership. The WebRTC foundation has peer/data-channel code, but lacks operational signaling, full negotiation, real room membership, and the depicted moderation system. Small-group safety also needs blocking, reporting, facilitator procedures, consent, and protection from identifying information—not just encryption.

Heritage Echoes: emotional connection across generations

This page proposes recording family stories, transcribing and translating Hindi and regional languages, and preserving emotionally tagged history. A DNA-helix canvas reinforces the inheritance metaphor. No recorder, transcription model, translator, consent workflow, or persistent family archive is connected to the page.

A future version must consider consent from every recorded participant, control over sharing and deletion, translation quality, and storage ownership. Family stories can contain information about people who never created an account.

The shared principle

These features expand the same core thesis: emotional wellbeing is not only a chat conversation. It can involve action, peers, and family history. The prototype communicates that product breadth successfully, while the next engineering phase should implement one safe, testable vertical slice before adding more ambitious promises.

Evidence: ResoQuests, Safe Circles, and Heritage Echoes pages; `domain/entities/ResonanceCircle.ts`; `infrastructure/webrtc/PeerTopologyMesh.ts`.

18 / PARSER, VIRTUAL FILES, AND STATE

The terminal: a project within the project

Why the terminal is special

The Terminal page is a substantial React component with its own types, phase machine, in-memory file tree, command dispatch, log rendering, scrambling, command history, and onboarding flow. It lets a visitor discover the product through interaction rather than only reading a marketing page.

The command model

Input is trimmed and split into arguments. The first token is normalized to lowercase and dispatched in a switch. Before ordinary dispatch, the current phase can intercept input as a name, email, or role. This is a small conversational state machine, not simply a collection of buttons styled like a console.

The command list includes `help`, `join`, `ls`, `cd`, `cat`, `scan`, `status`, `decrypt`, `ping`, `whoami`, `clear`, `history`, `hack`, `reboot`, `motd`, `nmap`, `trace`, `matrix`, and `exec`. These operate within the page's own logic; they do not give a visitor a real operating-system shell.

Virtual filesystem

The `FS` object holds a `home` tree with a README, manifesto, targets log, architecture documents, founder files, and a mock executable. A path array and `getDir()` support browsing. `cat` displays embedded strings, not arbitrary server files. This makes exploration safe from actual shell execution while preserving the fantasy of a system console.

Rendering and feedback

Typed log entries carry colors and animation flags. `ScrambleLine` decodes text from randomized characters. Timed log batches create scans and boot sequences. The page also uses acoustic cues and busy phases to sequence output. Simulated scan targets, network traces, and decryption messages are theatrical interface content, not actual security tooling.

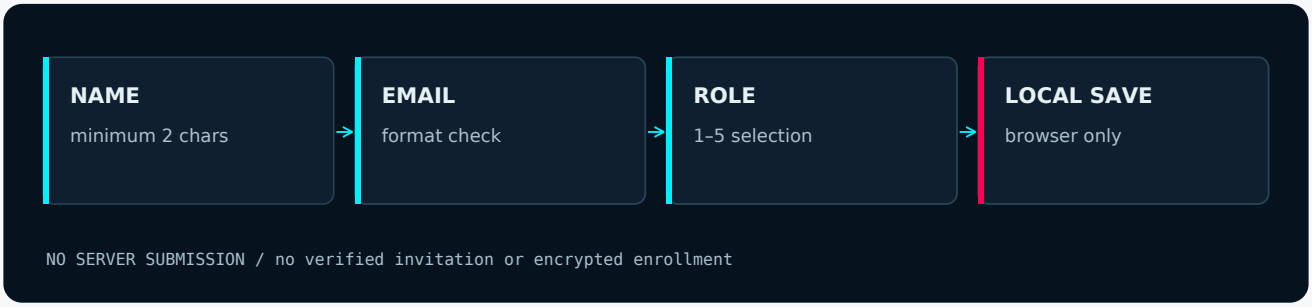
A good demo sequence

Type `help`, then `ls`. Read `manifesto.txt` with `cat`, enter the architecture directory with `cd architecture`, inspect its files, and use `cd ..` to return. Explain that this is a virtual product-documentation filesystem. Then demonstrate `join` using disposable example information, not sensitive personal data.

Evidence: `src/app/terminal/page.tsx`, including `FS`, `Phase`, `ALL_COMMANDS`, `ScrambleLine`, and `runCommand`.

19 / STORAGE AND CONVERSION REALITY

What happens when someone joins?



The terminal onboarding flow

`join` begins a sequence: name, email, role, completion. The name must be at least two characters and is converted to uppercase. The email is checked with a simple format expression and lowercased. The role is selected from Architect, Builder, Educator, Student, and Investor using numbers 1–5.

At completion the code generates a random queue position between 300 and 899, a random dotted node address, and a display access code. These are not authoritative assignments from a server and are not cryptographically secure credentials.

What is actually saved

The page reads the browser key `resonate_waitlist`, parses an array or starts with an empty one, appends name/email/role plus a timestamp, queue position, and node address, and writes it back with `localStorage.setItem`. Errors are swallowed by a `try/catch`. There is no network submission or guaranteed durable enrollment.

The stored object does not include a server-issued account, a verified email, a real invitation, or a backed-up registration. Clearing site storage removes it; another browser has a separate copy. Other scripts running under the same origin can access localStorage, so it is not appropriate to describe this record as end-to-end encrypted.

The separate Join page

`/join` contains native required inputs and an age range of 13–18. It has no `onSubmit`, API call, or implemented queue encryption. Because the controls also lack `name` attributes, native form submission is not a meaningful registration mechanism here. It is a visual form prototype.

How to make conversion real

Define a consent-aware waitlist contract, implement a server endpoint with schema validation and rate limits, deduplicate entries, verify email when appropriate, return an actual reference, and show explicit success/failure states. Collect only necessary information, document retention, and provide deletion. For minors, review the applicable consent and safeguarding requirements before gathering real data.

Evidence: Terminal's `join_name`, `join_email`, and `join_role` branches; `src/app/join/page.tsx`; API specification currently containing only health operations.

20 / MEANING BEFORE INFRASTRUCTURE

Domain design and shared state

Entity: stable identity and domain events

`Entity<T>` gives objects a stable `_id`, defaulting to `crypto.randomUUID()`, plus public properties and a private event list. Subclasses can append events; callers can inspect and clear them. This separates “something meaningful happened” from whatever external action may later respond to it.

User and emotional state

`User.create()` refuses an empty `cryptoPublicKey`. `registerEmotionalShift()` updates the baseline and last-active date, and queues an event for a critical state. `completeResoQuest()` increments a session streak and queues a completion event. No persistence layer or frontend workflow automatically follows from these methods.

`EmotionalState` extends a shallow-frozen ValueObject. It checks that valence is between −1 and 1. Criticality is encoded as valence below −0.8 and arousal above 0.8; intervention is flagged when a distortion is marked and valence is below −0.5. These thresholds are developer-authored heuristics, not clinically validated screening criteria. Arousal/dominance bounds and finite-number checks also need validation.

ResonanceCircle: intended invariants

The circle entity stores topic, capacity, participant IDs, encryption/moderator fields, and creation time. It rejects capacities above six, checks fullness when joining, and emits join/leave events. The full-capacity check occurs before duplicate-member detection, so an already-present participant can receive a capacity error. Minimum capacity and initial participant consistency are not fully enforced.

EventBus: dispatch, not full CQRS

A singleton maps event names to asynchronous handlers. Publishing executes handlers for each event concurrently using `Promise.all`. The base event interface omits `type`, leading to `@ts-ignore` in event usage. A discriminated union would make event contracts explicit. There is no durable event store, command pipeline, or separate read model establishing a full CQRS system.

Zustand store

`useEmpathyStore` holds a mood score, distortion labels, and resonance points initialized to 120. It supports updates, deduplication, clearing, and awards. It has no persistence middleware or automatic binding to the domain model. Think of it as a lightweight state foundation, not a complete application database.

Evidence: `core/cqrs/`; `domain/`; `presentation/store/empathyStore.ts`.

Offline-first synchronization

Why a vector clock exists

Offline devices can edit independently. A single local counter cannot tell whether one edit happened after another or whether both happened separately. A vector clock tracks counters by client ID. Local writes increment the current client's counter; merging clocks takes the maximum count for each client.

A concrete example

Device A writes with `{A: 1, B: 0}` while device B independently writes with `{A: 0, B: 1}`. Neither clock dominates the other: each has a counter the other lacks. They are concurrent. If A later receives B's edit and writes again, a suitably merged clock could become `{A: 2, B: 1}`, which is causally newer than either earlier clock.

`compare()` returns 1 when the local clock dominates, −1 when it is dominated, and 0 for either concurrency or equality. Callers must remember that zero does not mean “identical.”

The LWWMap implementation

Each map entry holds a value, wall-clock timestamp, and VectorClock. `set()` clones an existing entry's counters, increments the local counter, and records `Date.now()`. `merge()` accepts missing entries or causally newer remote entries. For concurrent/equal clocks it falls back to the newer timestamp; exact timestamp ties use a client-ID string comparison. `serialize()` exports values, timestamps, and clock counters as JSON.

Why this is only a foundation

The map does not provide a network transport, IndexedDB storage, operation log, journal UI binding, deserializer, or deletion tombstones. Device clocks can disagree. The code also replaces entries rather than consistently merging causal context from losing concurrent writes; tie behavior and ownership of clock client IDs deserve convergence tests. The serialization format does not separately preserve every runtime clock object's client identity.

A last-writer-wins policy can discard meaningful text. A real journal may need a text-specific CRDT or an explicit user-visible conflict policy, not silent whole-entry replacement. A service worker and offline asset cache would also be needed for a reliable offline web experience after fresh navigation.

Accurate presentation language

“I implemented vector-clock comparison and a draft LWW merge strategy to explore offline-first synchronization. A complete conflict-free journal still needs persistence, transport, recovery, and multi-device convergence tests.”

Evidence: `core/crdt/VectorClock.ts`; `/crdt` page. No integrated production synchronization is established by this source file alone.

The AI foundations, unpacked

The worker is a lexical scoring demo

The worker lowercases a supplied string, splits it into words, and adds scores from a small dictionary. Terms such as “worthless” and “panic” have negative weights; “calm,” “hope,” and “strong” have positive ones. It returns the score, matched words, a timestamp, and a threshold-based state label.

A score below −5 becomes `CRITICAL_INTERVENTION`; a negative score becomes `COGNITIVE_DISTORTION`; otherwise it becomes `STABLE_BASELINE`. A million-iteration square-root loop is deliberately included to simulate heavy computation. This is not useful inference and should be removed for real deployment.

The worker demonstrates moving work off the UI thread, but does not understand negation, sarcasm, multilingual text, context, or clinical risk. “Not anxious” can still match “anxious.” It is not evidence of a dependable crisis detector.

The WebGPU node is a compute primitive

`NeuralComputeNode` checks `navigator.gpu`, requests an adapter/device, and creates a WGSL compute pipeline. The shader performs an elementwise operation:

```
output[i] = 1 / (1 + exp(-(input[i] * weight[i])))
```

The compute method allocates storage buffers, uploads input/weights, creates a bind group, dispatches workgroups of 64, copies the result into a readable buffer, maps it, copies the values, and destroys buffers on the success path.

What it does not contain

There is no tokenizer, trained weight artifact, Whisper decoder, embedding model, sequence architecture, or LLM inference engine. Unsupported WebGPU returns false with a fallback warning, but this class does not itself wire up the worker. Inputs need shape checks; typed-array offsets, zero lengths, device loss, mapping errors, and failure-path cleanup need attention.

The right explanation

“The code explores two execution paths: a CPU worker for lexical scoring and a GPU kernel for parallel numeric transforms. Neither should be presented as the finished empathy model. The product's AI claims remain a future integration and evaluation task.”

Evidence: `workers/sentiment.worker.ts`; `infrastructure/ai/webgpu/NeuralComputeNode.ts`; chat and Echo Studio source. The worker path name is verified in the repository inventory.

23 / REAL PRIMITIVES, INCOMPLETE SYSTEM

Privacy and peer networking

CipherProtocol: what is implemented

The helper derives a non-extractable 256-bit AES-GCM key from a password using PBKDF2 with SHA-256, a random 16-byte salt, and 100,000 iterations. Encryption generates a fresh random 12-byte IV, encodes plaintext, and calls Web Crypto. Decryption accepts ciphertext, key, and IV and decodes the result.

These are meaningful cryptographic operations. They do not automatically encrypt chat state, terminal records, every request, or a database. A complete design needs key recovery, device migration, versioned encrypted storage, authenticated metadata where appropriate, a calibrated password KDF policy, and safe consent-based handling of sensitive data. Endpoint compromise can expose plaintext even if network encryption is sound.

PeerTopologyMesh: what is implemented

The class holds peer connections, data channels, and an optional microphone stream. It specifies two Google STUN servers, requests audio-only media, adds local tracks, creates peer connections/data channels, emits placeholder signals, broadcasts JSON to open channels, and can close peers or stop all local tracks.

What is missing for actual circles

`emitSignal()` is a placeholder. There is no operational signaling service, complete offer/answer negotiation, remote-description/candidate exchange flow, TURN relay configuration, membership authentication, incoming-channel integration, or rendered incoming audio. Logging an incoming stream is not a group conversation.

WebRTC transport encryption is useful but does not prove user anonymity or a zero-knowledge application. Signaling and ICE can expose metadata. A mesh also grows quickly: n participants require up to n(n−1)/2 pair links, or 15 at six participants and 28 at eight. This reinforces the small-group design while making mobile testing essential.

Resolve the privacy contradiction

The policy copy promises anonymity while also promising emergency identification or guardian contact. These need an explicit, feasible design with informed consent and limited access—not a claim that anonymity can be “bypassed” automatically. Safeguarding requirements must be agreed with qualified humans before deployment to minors.

Evidence: `infrastructure/crypto/CipherProtocol.ts`; `infrastructure/webrtc/PeerTopologyMesh.ts`; Privacy, Protocol, and Safe Circles pages.

24 / SUPPORT SYSTEMS AND SCAFFOLDING

Telemetry, memory, and the backend

Telemetry is a mock buffer

`AnonymizedMetrics` stores events in a static array and flushes at 50 entries. Metadata values typed as numbers or booleans are copied while strings are dropped. Flush clears the buffer and waits 100 milliseconds; it does not post to Datadog or another collector.

Removing strings is not a proof of anonymization: timestamps, numeric identifiers, combinations of attributes, and event names can still identify people. A real telemetry design needs an allowlist, consent/retention rules, privacy review, batching guarantees, and explicit failure behavior. The `/telemetry` page's operational numbers are separate hard-coded display values.

MemoryManager is not a replacement garbage collector

The manager tracks WeakRefs, checks the nonstandard `performance.memory` field about every ten seconds, and sweeps above a 50 MB threshold. It removes dead references and calls `destroy()` on objects marked stale. It cannot force JavaScript's garbage collector or guarantee a memory ceiling. Explicitly canceling timers, stopping media tracks, disconnecting audio nodes, and disposing GPU resources remain necessary.

The API actually exposes one operation

Express mounts its router under `/api`. The health route is `GET /api/healthz`; it validates `{ status: "ok" }` through generated Zod code and returns JSON. The app uses JSON/form parsers, CORS, and Pino request logging that omits query strings. The server requires a positive numeric `PORT`.

There are no account, journal, waitlist, circle, model-inference, or SOS endpoints. The generated React health hook is a contract example, not a full product API.

Database readiness

The database package creates a PostgreSQL pool and Drizzle instance and throws if imported without `DATABASE_URL`. Its schema exports no tables. Declaring the package as an API dependency does not mean the active health route needs a database connection. No journal persistence, user schema, migrations, or audit log is implemented in the empty schema.

Evidence: telemetry and memory modules; `artifacts/api-server/src/`; `lib/api-spec/openapi.yaml`; `lib/db/src/`; generated API libraries.

How the monorepo holds together

Why pnpm workspaces?

The workspace globs include `artifacts/*`, `lib/*`, integration packages, and scripts. Nine workspace projects were recognized during installation. The catalog centralizes common versions such as React, Vite, Tailwind, Zod, and React Query. `workspace:*` links internal libraries without publishing them.

The root `preinstall` script refuses non-pnpm installation and removes competing npm/yarn lockfiles. `pnpm-lock.yaml` records resolved dependencies. The package-release-age setting is 1,440 minutes with limited exclusions; built-dependency allowlists and overrides constrain parts of installation. These are supply-chain precautions, not a security audit.

The contract-generation chain

`lib/api-spec/openapi.yaml` defines the health operation. Orval reads the spec and generates a React Query client plus Zod validation/types using its configuration. The custom fetch wrapper supports common transport concerns such as base URLs, headers, response parsing, and errors. Generated code should normally be updated through the spec/codegen process, not edited ad hoc.

TypeScript boundaries

The base config targets ES2022, uses bundler resolution, enables project-oriented checks, and turns on several strict options. The frontend adds DOM libraries and the `@/*` source alias. Project references help typecheck shared libraries before artifacts. The `@assets` Vite alias points at attached repository images; TypeScript configuration is not identical to every Vite resolution rule.

UI libraries and development sandbox

The Radix-based UI directory includes dialogs, inputs, menus, tables, sidebars, charts, and more. Companion dependencies include React Hook Form, resolvers, date-fns, Day Picker, Recharts, Embla, Sonner, Vaul, and class-name utilities. Many are scaffold/library support rather than essential to the bespoke page designs.

The mockup sandbox discovers preview components using a Vite plugin and a generated module map. It is useful for isolated component work; it is not a hidden wellbeing backend. `scripts/src/hello.ts` is a simple utility example. Replit's `scripts/post-merge.sh` installs frozen dependencies and invokes the database push command; because that can change a provisioned schema, it needs deliberate environment configuration and should not be run casually for documentation.

Evidence: workspace YAML; manifests; root/base TypeScript configs; `lib/api-spec/orval.config.ts`; API client; mockup plugin; `.replit`; `scripts/`.

26 / MIGRATION HISTORY AND STALE ASSUMPTIONS

What changed from Next.js

The preserved original

`.migration-backup/` contains the prior Next.js application, its package files, configuration, ideas document, and source tree. Its manifest declares **Next.js 16.2.9**, React 19.2.4, Anime.js 3, and type packages for Three/WebGPU. The current frontend no longer runs that application's Next build or routing.

The migration described by repository notes

`replit.md` says the project was migrated because the selected Replit artifact workflow did not support Next.js artifacts. This is a documented project-specific tooling constraint, not a statement that Next.js cannot run anywhere on Replit.

The current implementation replaces Next Link/location APIs with Wouter, moves the app shell into `App.tsx`, loads fonts via HTML links instead of `next/font`, loads shader source through a Vite plugin, and updates the active loader to Anime.js 4's named `animate`, `ease`, and `onComplete` conventions.

What the migration preserved

The route component directory layout, visual identity, canvas scenes, most presentation concepts, and the core/domain/infrastructure split remain recognizable. A backup helps explain the evolution, but the active app must be read from `artifacts/resonate/`, not assumed from the old README.

What the migration left behind

The Technology page still advertises “Next.js 15” edge rendering. The active Vite app is a client-side SPA, so those SSR/edge claims are stale. A duplicate `components/animations/LoadingScreen.tsx` still imports Anime.js as a default export and fails typechecking against v4. Duplicate legacy layout/canvas folders and CSS files can also confuse maintainers.

The current manifest lacks direct Three/WebGPU type coverage needed by active source checks and has an unresolved `postprocessing` import in the singularity source. This helps explain why a production bundle can succeed while a repository-wide typecheck fails.

The maintainability lesson

Migration is more than getting a dev server to render. Complete it by reconciling dependencies, pruning or clearly labeling inactive duplicates, updating product copy, checking route/deep-link behavior, and verifying build/typecheck parity. This document records the state rather than silently changing application behavior.

Evidence: `.migration-backup/package.json`; `replit.md`; current App/Vite files; duplicate LoadingScreen; `/technology`; validation logs.

27 / REPRODUCIBLE COMMANDS AND BOUNDARIES

Run, build, and deploy it

Install and launch the frontend

Use Node.js with pnpm. The Replit environment declares Node 24; this inspection used Node 22.22.3 and pnpm 10.32.1 successfully for install and the targeted frontend bundle. Use the repository lockfile rather than changing dependency versions just to make documentation.

```
pnpm install --frozen-lockfile
pnpm --filter @workspace/resonate run dev
```

Vite defaults to port 5173, accepts a positive `PORT` override, binds to `0.0.0.0`, and allows preview hosts. `BASE_PATH` controls the asset/router base. External Google Fonts require network access; system fallbacks remain important in restricted environments.

Validation and targeted output

```
pnpm run typecheck
pnpm --filter @workspace/resonate run build
pnpm --filter @workspace/resonate run serve
```

The targeted build writes `artifacts/resonate/dist`. The root `pnpm run build` first runs the full typecheck, then recursive package builds. It should currently be expected to stop at the known frontend type errors; do not equate the targeted Vite success with root-build success.

Optional API and code generation

```
PORT=8081 pnpm --filter @workspace/api-server run dev
pnpm --filter @workspace/api-spec run codegen
```

If using the database package, provision PostgreSQL and supply `DATABASE_URL`; do not put credentials in source. There is no frontend dev proxy to `/api` in the present Vite config. Future browser calls should use relative URLs with a same-origin proxy or explicitly configured deployment routing, not sandbox-local hostnames.

Deployment caveat

The root `vercel.json` declares framework `vite`, build `pnpm run build`, output `dist`, and an SPA rewrite to `/index.html`. Its baseline commit message explicitly refers to a Root Directory setting. Root directory selection is therefore essential: deploying from the repository root differs from deploying the frontend artifact. Verify which config is applied and where output is produced. The rewrite alone does not deploy the Express service.

Evidence: `.replit`; package scripts; Vite config; root `vercel.json`; commit `14fe277`. Deployment itself was not executed during this documentation task.

28 / BUILD EVIDENCE AND CURRENT ISSUES

What I verified

Checks executed on 18 October 2026

Dependency install — passed. `pnpm install --frozen-lockfile` recognized nine workspace projects and installed 545 packages. No application dependency or lockfile change was required.

Targeted frontend production bundle — passed. `pnpm --filter @workspace/resonate run build` used Vite 7.3.3, transformed 2,253 modules, and completed in about 14.7 seconds in this environment. Reported output was approximately 1,560.81 kB of JavaScript, or 457.21 kB gzip, plus 111.90 kB of CSS, or 18.31 kB gzip. These are build output measurements, not performance benchmarks on users' devices.

Workspace typecheck — failed. Shared-library checks and API/mockup/scripts checks completed, but the frontend reported TypeScript errors. Important categories:

- Terminal's Framer Motion easing string `"steps(1)"` is not accepted by its typed easing API.
- The legacy LoadingScreen imports Anime.js as a default export that v4 does not provide.
- Multiple canvas files lack Three declaration coverage.
- WebGPU types such as GPUDevice, GPUBufferUsage, GPUMapMode, and navigator.gpu are missing from type context.
- SingularityCanvas cannot resolve a direct `postprocessing` module/type import.
- RoadmapCanvas includes implicit-any callback parameters.

Why one check passed and the other failed

Vite bundles and transpiles the active import graph; it does not perform the same semantic checking as `tsc` across every included source file. Some problematic files are inactive duplicates or not in the production import graph. Therefore, successful output does not imply all source is type-correct.

Prioritized response

First reconcile imports, declarations, and motion typing; then make the root build green. Next add route-level splitting, inspect bundle composition, and test route navigation, mobile layout, sound consent, keyboard access, and WebGL fallback in actual browsers. The build warns about chunks over 500 kB; simply raising the warning threshold is not an optimization.

No browser end-to-end suite, live peer-network test, clinical evaluation, database integration test, or deployment verification was performed. The walkthrough is transparent about that scope. Raw validation output is included in the supporting evidence directory.

Evidence: executed install/build/typecheck commands; `walkthrough/evidence/`; active manifest and TypeScript configuration.

What I did not use—or finish

Not the current runtime stack

The active app does not use Next.js routing or SSR, despite retained directories and stale copy. It is not an Angular/Vue app, a native mobile app, or a Python AI service. Expo-related comments in workspace versions do not establish an implemented mobile product. The repo declares PostgreSQL plumbing, not a completed database-backed feature set.

Not an integrated AI product yet

No trained Whisper model, LLM checkpoint, inference runtime, dataset, training pipeline, reinforcement-learning policy, or evaluation suite is supplied for the advertised feature flows. The chat does not call OpenAI or a local model; the journal simulation does not capture microphone audio. The WGSL shader's sigmoid arithmetic is not a language model.

Not a complete identity or security system

There is no operational login/session service, email verification, password/account recovery, full key exchange, audited E2E messaging, zero-knowledge proof system, or production authorization layer. The terminal's “hack” and “scan” commands are simulations. Its random access code is not an authentication token.

Not live operational evidence

Home's memory display, Mood Genome cells, Telemetry values, impact numbers, and roadmap statuses do not establish actual user counts, health outcomes, latency, uptime, or completed school deployments. Home's “42%,” “85%,” and “0 MS” statements need sourcing or relabeling. “Zero latency,” “military-grade,” and “absolute anonymity” are overclaims, not verified properties.

Not a medical or emergency service

The UI includes a medical disclaimer, but the scripted chat and heuristic thresholds are not appropriate for real crisis triage. Automatic guardian escalation is described, not implemented. Safety cannot be delegated to a disclaimer or a classifier name.

Why include this in a portfolio walkthrough?

Honesty gives the project a strong engineering narrative: a finished visual prototype, deliberate architectural experiments, identified constraints, and a realistic plan. You can take full credit for the source work without claiming nonexistent deployment, scientific validation, or integrations. A reviewer can then evaluate the actual achievement on its merits.

Evidence: active feature handlers; API specification; empty DB schema; infrastructure methods; route content; validation results.

30 / A PRACTICAL IMPLEMENTATION ROADMAP

From prototype to responsible product

Stage 1 — Make the current artifact trustworthy

Fix the known type errors and achieve a green root build. Reconcile active/legacy files and copy. Label all sample metrics, scripted AI output, and local-only enrollment. Add skip/mute/reduced-motion controls, keyboard navigation, touch-friendly menus, and high-contrast body text. Split routes and defer heavy 3D scenes.

Stage 2 — Ship one complete vertical slice

Start with consent-based text journaling rather than every advertised feature. Specify records, timestamps, deletion, storage encryption, recovery boundaries, and error states. Connect the UI to a small application-service layer and test it from input to persistence to retrieval. Only collect the minimum data needed.

Stage 3 — Add evaluated local assistance

Choose a real model/runtime based on device capability, download size, latency, multilingual quality, and privacy needs. Implement loading, cancellation, fallbacks, and a clear “this is an automated suggestion” label. Evaluate false positives/negatives, uncertainty, harmful responses, and culturally specific contexts with qualified review. Add voice capture only with explicit recording controls and deletion options.

Stage 4 — Add reliable collaboration

Implement authenticated signaling, peer negotiation, TURN fallbacks, reconnect behavior, membership rules, and participant controls. Decide whether a mesh is appropriate for the target devices. Test offline journals with persistence, deterministic conflict rules, device clock skew, repeated merges, and recovery after partial failure.

Stage 5 — Pilot with human safeguards

Before involving real minors, settle consent, age handling, privacy policy, moderation staffing, escalation workflows, and incident response with qualified advisors. Make human help accessible without relying on a speculative automated override. Do not run unsupervised crisis experiments.

Acceptance criteria beat slogans

- All explicit routes load and unknown routes show a usable 404.
- Root typecheck/build pass; no critical console or accessibility errors.
- A journal can be saved, reopened, exported/deleted, and recovered as specified.
- Encryption and sync have automated tests and documented limitations.
- Mobile performance is measured on representative low-end hardware.
- Every displayed statistic has a source or a visible “demo” label.
- Support and privacy language matches the behavior actually delivered.

Evidence basis: current validation failures and feature boundaries. These are recommendations, not a claim that the listed future work has been completed.

31 / A FIVE-MINUTE WALKTHROUGH SCRIPT

How to present RESONATE

0:00–0:40 — Problem and purpose

“RESONATE is my prototype for an algorithmic sanctuary for Indian students. The idea responds to academic pressure, loneliness, and shallow online connection. I wanted to demonstrate how technology could prioritize reflection and meaningful support rather than endless engagement.”

0:40–1:20 — Experience and identity

“I expressed the concept as a cinematic system: black and purple backgrounds, cyan for connection, magenta for interruption, terminal typography, procedural sound, and GPU-rendered particles. The hero is not a video; it uses 60,000 points with custom GLSL shaders that respond to time and pointer movement.”

1:20–2:10 — Feature demonstrations

“Echo Studio demonstrates the voice-journal journey: transcript, possible distortion, and a balanced reframe. The current example is scripted. Empathy Coach is also a scripted chat prototype, while Mood Genome demonstrates how a future dashboard could show patterns. I distinguish these interfaces from the model integration still required.”

2:10–3:00 — The interactive terminal

“This terminal is implemented in React with a command parser, virtual filesystem, history, animated logs, and an onboarding state machine. You can browse product documentation and run `join`. Its waitlist is currently saved in this browser's localStorage, not sent to an enrollment server.”

3:00–3:50 — Engineering underneath

“The active stack is React, TypeScript, Vite, Tailwind, and Wouter in a pnpm monorepo. Three/Fiber, GSAP, Framer Motion, Anime.js, Lenis, and Web Audio shape the experience. I also separated domain entities, vector clocks, browser cryptography, WebRTC, and GPU/worker experiments as foundations for a future local-first product.”

3:50–4:30 — Evidence and limitations

“The frontend Vite bundle succeeds, but the workspace TypeScript check identifies issues I have documented. The product does not yet have a deployed LLM, real Whisper transcription, a complete peer network, or verified impact metrics. My next step is to complete one safe, persistent, evaluated journal workflow.”

4:30–5:00 — Close

“The achievement is a coherent, technically ambitious portfolio prototype that makes a social idea tangible. It shows my ability to combine visual engineering, frontend interactions, architectural exploration, and honest analysis of what production readiness requires.”

Presenter note: use your own voice and only claim your personal contribution where accurate. The repository identifies founders, but it does not establish every individual's authorship history.

32 / INTERVIEW AND VIVA PREPARATION

Questions you should be able to answer

Why so many animation libraries?

“They serve different roles: GSAP for imperative scroll work, Framer Motion for state-linked UI, Anime.js for the loader transition, and Lenis for scroll feel. This creates flexibility but also bundle and lifecycle cost. A production cleanup could consolidate overlapping responsibilities.”

Why use Vite instead of Next.js?

“The repo notes record a migration to fit the chosen Replit artifact workflow. The active result is a React SPA with Wouter. The backup preserves the earlier Next.js implementation, but I do not claim SSR benefits for the current app.”

Does GPU compute make the AI private and instant?

“No. Local execution can reduce server transmission, but privacy depends on the entire data lifecycle. Hardware work still takes time. The current GPU module only demonstrates elementwise sigmoid computation, not a complete language model.”

Is the app actually end-to-end encrypted?

“AES-GCM/PBKDF2 helper code exists, but the current chat and waitlist flows do not integrate it. Full E2E requires identity/key management, transport design, storage handling, and testing; the visual badge is not sufficient.”

Why not just use a cloud chatbot?

“The concept prioritizes sensitive student data and potential offline resilience. Local-first processing is an intentional direction, but model quality, hardware limits, download size, and safe fallback behavior must be evaluated before deciding the final architecture.”

What is your strongest completed work?

“The coherent interactive frontend: 26 explicit routes, reusable motion and canvas elements, custom shaders, procedural sound, a stateful terminal, and a consistent visual narrative. The prototype also surfaces the engineering boundaries for a future product.”

What would you fix first?

“Typecheck/build parity, misleading copy, accessible navigation and loading controls, and initial-bundle size. Then I would implement a complete journal slice before expanding the feature count.”

Are the founders' roles consistent everywhere?

“The active Founders/Terminal pages describe Anuj as Developer/AI/Backend and Aarav as Vision/Marketing/Ideas. The older ideas file assigns leadership and AI roles differently. I would treat active profiles as the current narrative and confirm personal credits directly rather than infer authorship from old planning text.”

Evidence: relevant implementation chapters and source files; baseline history is shallow and cannot prove a detailed timeline of individual contributions.

Glossary and final perspective

The key vocabulary

SPA: a client application whose router changes displayed views without a full server-rendered document per route. **SSR:** generating page HTML on a server; not provided by this Vite frontend. **Uniform:** a shader input shared across many GPU executions, such as elapsed time or mouse position. **Attribute:** per-vertex/particle data such as position or randomness.

GLSL: shader language used by the visual rendering programs here. **WGSL:** WebGPU shader language used by the experimental compute node. **Web Worker:** an execution context separate from the main UI thread. **Lexical scoring:** assigning values to matching words; not the same as language understanding.

Entity: a domain object with stable identity. **Value object:** an object defined mainly by its values. **Domain event:** a record of a meaningful domain change. **CQRS:** separating command/write and query/read responsibilities; an event bus folder alone does not implement the full pattern.

Vector clock: per-client counters that help compare causal histories. **LWW:** last-writer-wins resolution, which selects a winner rather than preserving every edit. **CRDT:** a data type designed to converge under specified replication/merge rules; those guarantees require careful implementation and testing.

AES-GCM: authenticated symmetric encryption. **PBKDF2:** a password-based key derivation function. **IV:** per-encryption initialization value; AES-GCM must avoid IV reuse with a key. **E2E:** plaintext is available only at intended endpoints under a defined threat model—not a synonym for HTTPS or a UI badge.

STUN: helps discover connectivity information. **TURN:** relays traffic when direct peer connections fail. **Signaling:** exchanges negotiation information so peers can connect. **Zero-knowledge:** a precise cryptographic or architectural claim requiring a defined mechanism, not a general adjective for privacy.

The takeaway

RESONATE's value is the combination of a clear problem statement, a distinctive visual identity, ambitious browser engineering, and a wide product vision. Its current code is best understood as an interactive portfolio and experience prototype with useful but incomplete systems foundations.

You can explain it powerfully without exaggeration: **“I made the idea tangible, built the interface and visual engine, explored the architecture, and identified exactly what remains before it can become a safe real-world product.”**

Evidence: terminology is applied to the inspected implementation. The appendices provide a complete source/dependency inventory and a study-oriented code-reading map.

A1 / A GUIDED SOURCE-STUDY MAP

Read the code in this order

Pass 1 — Understand the actual runtime

Read `artifacts/resonate/index.html`, `src/main.tsx`, and `src/App.tsx`. Trace the root element, CSS import, provider nesting, loader gate, and explicit route switch. Then read `vite.config.ts` to understand aliases, shader imports, ports, and output paths. This prevents treating the preserved Next.js structure as active framework behavior.

Pass 2 — Follow a real user event

Read `presentation/layouts/AppProvider.tsx`, `presentation/components/animations/CinematicLoader.tsx`, and `infrastructure/audio/AcousticResonator.ts`. Follow one click from state change through AudioContext initialization, timers, Anime.js transition, and route-content mount. Note exactly which timers and resources have cleanup.

Pass 3 — Study the most complete interactive page

Read Terminal's types and `FS`, then `runCommand`, onboarding branches, keyboard handler, and log renderer. Draw the phases on paper. Follow a successful and an invalid join input. Search for `localStorage` and identify the actual record rather than repeating the success message.

Pass 4 — Trace one graphics frame

Read `NeuralNetworkCanvas.tsx`, `neural.vert`, and `neural.frag`. Identify geometry attributes, uniforms, material settings, pointer normalization, and the `useFrame` update. Compare with FluidCanvas, where JavaScript updates thousands of instances each frame instead of moving the whole animation into a vertex shader.

Pass 5 — Separate demo from infrastructure

Read Echo Studio and AdvancedChat before reading NeuralComputeNode and the sentiment worker. Search for calls connecting them: a nearby folder or similarly named concept does not establish an integration. Repeat this check for CipherProtocol, PeerTopologyMesh, EventBus, Zustand, and LWWMap.

Pass 6 — Inspect backend contracts and evidence

Read `lib/api-spec/openapi.yaml`, generated health client/validator, Express routing, and the empty DB schema. Finish with the build/typecheck logs. Compare every marketing guarantee with the implementation it would require.

Evidence: all paths above are present in the baseline; the next appendices index the full tracked repository and declared package set.

A2 / WHAT THE NAMES REALLY MEAN

More scenes, under the microscope

Not all particle scenes use the same technique

CinematicLoaderCanvas uses 25,000 colored points in a tunnel-like distribution. MeshTopologyCanvas builds a sphere-like arrangement of nodes and connecting line segments. DNAHelixCanvas uses 3,000 points in two alternating strands with time-varying movement. These are mathematical visualizations, not network or genomic data.

GlobeCanvas distributes 3,000 points over a sphere using a formula based on index/angle. It colors a selected region and rotates the scene. The “impact globe” does not fetch a live map of students. VectorFieldCanvas draws an instanced cone grid whose orientations respond to pointer distance. It illustrates a field but is not the vector-clock implementation.

FluidCanvas is CPU-updated instancing

The fluid scene uses 10,000 instances with randomized motion parameters. Each frame iterates the particles, computes trigonometric positions and orientation, updates matrices/colors, and marks instance buffers dirty. Instancing reduces draw-call overhead, but these per-instance calculations still run in JavaScript. Its comment about “WGSL-like logic” does not make it a WebGPU compute shader.

Other specialized scenes

AudioVisualizerCanvas deforms a plane using layered sine/cosine waves and a center pulse. It does not read microphone spectrum data or an AnalyserNode. RoadmapCanvas creates a Catmull–Rom spline and maps scroll progress to camera movement along negative Z. PrismCanvas rotates an icosahedron with custom glass shaders and an inner dark core. SingularityCanvas is an elaborate 150,000-point experimental scene with postprocessing imports and pointer-velocity input.

A concrete performance improvement

FluidCanvas allocates Color objects inside its per-particle frame loop. Reusing scratch objects and precomputed endpoint colors would reduce allocation pressure. Scene work should also pause when offscreen or hidden, scale with device capability, and dispose resources reliably. Do not use a synthetic animated “memory” label as a substitute for profiling.

Shader path audit

PrismCanvas imports `../shaders/glass.*` even though it lives in the nested `canvas/premium` directory. That relative path does not point to the sibling presentation shader directory. It is an inactive-source maintenance hazard: importing that component into a live route would require fixing resolution and testing it. Successful current bundling does not validate every unreferenced scene.

Evidence: all eleven files in `src/presentation/canvas/`, including nested scene directories; custom shader import paths; build log.

A3 / FROM SOURCE INSPECTION TO DEPENDABLE BEHAVIOR

A test plan you can explain

Unit tests for the foundations

Test EmotionalState boundaries, NaN/nonfinite input, and arousal/dominance validation. Test User event payloads, stable identity, and event clearing. Test circles at zero/full capacity, duplicate joins, repeated leaves, and inconsistent initial participants. Replace event `@ts-ignore` contracts with a typed union before relying on handlers.

For clocks, test equality, dominance, concurrency, partial client sets, and repeated merges. Property-style tests should check commutativity, associativity, and idempotence under the intended merge contract. Add clock skew, tied timestamps, serialization round-trips, and deletion behavior. A passing example is not a convergence proof.

Integration tests for sensitive operations

Test encryption/decryption round-trip, wrong keys, altered ciphertext, fresh IVs, stored salt/IV encoding, and failure recovery. Test GPU input lengths and unsupported-device behavior separately from model accuracy. Test peer negotiation with real signaling, disconnect/reconnect, permission denial, and relay-only connectivity once those features exist.

Browser tests for the existing prototype

Verify all 26 routes and direct deep links, the 404, menu focus, touch navigation, context-menu edges, and base-path deployment. Exercise loader completion and unavailable audio. Confirm transcript timing and scripted chat behavior; use fake timers to catch callbacks surviving unmount. Verify terminal input validation, command history, path navigation, and blocked/quota-full localStorage.

Performance and accessibility checks

Measure initial transfer, parse/execute time, long tasks, frame rate, and memory on representative low-end phones. Check re-entry after route changes. Audit focus order, names/labels, contrast, heading hierarchy, reduced motion, zoom, and screen-reader announcement of streaming logs. The viewport's `maximum-scale=1` deserves removal/review to support user zoom.

What is not already proven

No test scripts or automated product test suite in the inspected manifests establish these guarantees today. The two executed validation commands check type correctness and bundle construction, not all runtime or safety behavior. Treat this page as an actionable plan.

Evidence basis: current source structure, lifecycle patterns, declarations, and recorded validation output. Proposed tests have not been claimed as executed.

B1 / ROOT / WORKSPACE AND DEPLOYMENT

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
.gitignore	49 lines
.npmrc	2 lines
.replit	32 lines
.replitignore	5 lines
package.json	16 lines
pnpm-lock.yaml	6,296 lines
pnpm-workspace.yaml	160 lines
replit.md	57 lines
tsconfig.base.json	26 lines
tsconfig.json	16 lines
vercel.json	12 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

B2 / FRONTEND / ARTIFACTS/RESONATE

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
.replit-artifact/artifact.toml	31 lines
components.json	20 lines
index.html	24 lines
package.json	87 lines
public/favicon.svg	3 lines
public/file.svg	1 lines
public/globe.svg	1 lines
public/next.svg	1 lines
public/opengraph.jpg	120,056 bytes
public/robots.txt	2 lines
public/vercel.svg	1 lines
public/window.svg	1 lines
src/App.tsx	93 lines
src/app/about/page.tsx	74 lines
src/app/archives/page.tsx	36 lines
src/app/blog/page.tsx	47 lines
src/app/careers/page.tsx	48 lines
src/app/crdt/page.tsx	40 lines
src/app/engine/page.tsx	116 lines
src/app/favicon.ico	25,931 bytes
src/app/features/echo-studio/page.tsx	97 lines
src/app/features/empathy-coach/page.tsx	44 lines
src/app/features/heritage-echoes/page.tsx	43 lines
src/app/features/mood-genome/page.tsx	64 lines
src/app/features/page.tsx	46 lines
src/app/features/resoquests/page.tsx	61 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
src/app/features/safe-circles/page.tsx	46 lines
src/app/founders/page.tsx	165 lines
src/app/globals.css	129 lines
src/app/impact/page.tsx	81 lines
src/app/join/page.tsx	56 lines
src/app/manifesto/page.tsx	53 lines
src/app/not-found.tsx	39 lines
src/app/page.tsx	195 lines
src/app/privacy/page.tsx	37 lines
src/app/protocol/page.tsx	37 lines
src/app/roadmap/page.tsx	109 lines
src/app/schools/page.tsx	78 lines
src/app/technology/page.tsx	73 lines
src/app/telemetry/page.tsx	42 lines
src/app/terminal/page.tsx	973 lines
src/app/terms/page.tsx	32 lines
src/components/animations/LoadingScreen.tsx	101 lines
src/components/canvas/HeroCanvas.tsx	68 lines
src/components/layout/AppProvider.tsx	17 lines
src/components/layout/LenisProvider.tsx	31 lines
src/core/cqrs/Entity.ts	31 lines
src/core/cqrs/EventBus.ts	35 lines
src/core/crdt/VectorClock.ts	121 lines
src/core/memory/GarbageCollector.ts	81 lines
src/domain/entities/ResonanceCircle.ts	60 lines
src/domain/entities/User.ts	48 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
src/domain/value-objects/EmotionalState.ts	45 lines
src/hooks/use-mobile.tsx	19 lines
src/hooks/use-toast.ts	191 lines
src/index.css	143 lines
src/infrastructure/ai/webgpu/NeuralComputeNode.ts	120 lines
src/infrastructure/audio/AcousticResonator.ts	148 lines
src/infrastructure/crypto/CipherProtocol.ts	63 lines
src/infrastructure/telemetry/AnonymizedMetrics.ts	45 lines
src/infrastructure/webrtc/PeerTopologyMesh.ts	95 lines
src/lib/utils.ts	6 lines
src/main.tsx	5 lines
src/pages/not-found.tsx	21 lines
src/presentation/canvas/CinematicLoaderCanvas.tsx	100 lines
src/presentation/canvas/MeshTopologyCanvas.tsx	101 lines
src/presentation/canvas/NeuralNetworkCanvas.tsx	93 lines
src/presentation/canvas/SingularityCanvas.tsx	144 lines
src/presentation/canvas/archives/DNAHelixCanvas.tsx	95 lines
src/presentation/canvas/audio/AudioVisualizerCanvas.tsx	60 lines
src/presentation/canvas/fluid/FluidCanvas.tsx	86 lines
src/presentation/canvas/impact/GlobeCanvas.tsx	94 lines
src/presentation/canvas/premium/PrismCanvas.tsx	59 lines
src/presentation/canvas/roadmap/RoadmapCanvas.tsx	76 lines
src/presentation/canvas/vector/VectorFieldCanvas.tsx	110 lines
src/presentation/components/CustomCursor.tsx	65 lines
src/presentation/components/MagneticElement.tsx	51 lines
src/presentation/components/MatrixRain.tsx	73 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

B5 / FRONTEND / ARTIFACTS/RESONATE · CONTINUED

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
src/presentation/components/PurgeScreen.tsx	81 lines
src/presentation/components/ScrambleText.tsx	43 lines
src/presentation/components/TextReveal.tsx	37 lines
src/presentation/components/animations/CinematicLoader.tsx	152 lines
src/presentation/components/chat/AdvancedChat.tsx	136 lines
src/presentation/components/navigation/GlobalNav.tsx	120 lines
src/presentation/components/transitions/TransitionShutter.tsx	60 lines
src/presentation/components/ui/CinematicText.tsx	41 lines
src/presentation/components/ui/ContextMenu.tsx	55 lines
src/presentation/hooks/useGSAPOrchestrator.ts	37 lines
src/presentation/layouts/AppProvider.tsx	20 lines
src/presentation/layouts/LenisProvider.tsx	31 lines
src/presentation/shaders/glass.frag	32 lines
src/presentation/shaders/glass.vert	12 lines
src/presentation/shaders/neural.frag	11 lines
src/presentation/shaders/neural.vert	132 lines
src/presentation/shaders/singularity.frag	13 lines
src/presentation/shaders/singularity.vert	76 lines
src/presentation/store/empathyStore.ts	29 lines
src/workers/sentiment.worker.ts	40 lines
tsconfig.json	24 lines
vite.config.ts	71 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
accordion.tsx	55 lines
alert-dialog.tsx	139 lines
alert.tsx	59 lines
aspect-ratio.tsx	5 lines
avatar.tsx	50 lines
badge.tsx	43 lines
breadcrumb.tsx	115 lines
button-group.tsx	83 lines
button.tsx	65 lines
calendar.tsx	213 lines
card.tsx	76 lines
carousel.tsx	260 lines
chart.tsx	367 lines
checkbox.tsx	28 lines
collapsible.tsx	11 lines
command.tsx	153 lines
context-menu.tsx	198 lines
dialog.tsx	120 lines
drawer.tsx	116 lines
dropdown-menu.tsx	201 lines
empty.tsx	104 lines
field.tsx	244 lines
form.tsx	176 lines
hover-card.tsx	27 lines
input-group.tsx	168 lines
input-otp.tsx	69 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

B7 / UI LIBRARY / ARTIFACTS/RESONATE/SRC/COMPONENTS/UI · CONTINUED

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
input.tsx	22 lines
item.tsx	193 lines
kbd.tsx	28 lines
label.tsx	26 lines
menubar.tsx	254 lines
navigation-menu.tsx	128 lines
pagination.tsx	117 lines
popover.tsx	31 lines
progress.tsx	28 lines
radio-group.tsx	42 lines
resizable.tsx	45 lines
scroll-area.tsx	46 lines
select.tsx	159 lines
separator.tsx	29 lines
sheet.tsx	140 lines
sidebar.tsx	727 lines
skeleton.tsx	15 lines
slider.tsx	26 lines
sonner.tsx	31 lines
spinner.tsx	16 lines
switch.tsx	27 lines
table.tsx	120 lines
tabs.tsx	53 lines
textarea.tsx	22 lines
toast.tsx	127 lines
toaster.tsx	33 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

B8 / UI LIBRARY / ARTIFACTS/RESONATE/SRC/COMPONENTS/UI · CONTINUED

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
toggle-group.tsx	61 lines
toggle.tsx	43 lines
tooltip.tsx	32 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

B9 / SERVICES + LIBRARIES / REPOSITORY-RELATIVE

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
artifacts/api-server/.replit-artifact/artifact.toml	32 lines
artifacts/api-server/build.mjs	126 lines
artifacts/api-server/package.json	32 lines
artifacts/api-server/src/app.ts	34 lines
artifacts/api-server/src/index.ts	25 lines
artifacts/api-server/src/lib/.gitkeep	0 lines
artifacts/api-server/src/lib/logger.ts	20 lines
artifacts/api-server/src/middlewares/.gitkeep	0 lines
artifacts/api-server/src/routes/health.ts	11 lines
artifacts/api-server/src/routes/index.ts	8 lines
artifacts/api-server/tsconfig.json	19 lines
lib/api-client-react/package.json	15 lines
lib/api-client-react/src/custom-fetch.ts	371 lines
lib/api-client-react/src/generated/api.schemas.ts	10 lines
lib/api-client-react/src/generated/api.ts	101 lines
lib/api-client-react/src/index.ts	4 lines
lib/api-client-react/tsconfig.json	12 lines
lib/api-spec/openapi.yaml	36 lines
lib/api-spec/orval.config.ts	72 lines
lib/api-spec/package.json	11 lines
lib/api-zod/package.json	12 lines
lib/api-zod/src/generated/api.ts	16 lines
lib/api-zod/src/generated/types/healthStatus.ts	11 lines
lib/api-zod/src/generated/types/index.ts	9 lines
lib/api-zod/src/index.ts	2 lines
lib/api-zod/tsconfig.json	11 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
lib/db/drizzle.config.ts	14 lines
lib/db/package.json	25 lines
lib/db/src/index.ts	16 lines
lib/db/src/schema/index.ts	20 lines
lib/db/tsconfig.json	12 lines
scripts/package.json	14 lines
scripts/post-merge.sh	4 lines
scripts/src/hello.ts	1 lines
scripts/tsconfig.json	9 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
.replit-artifact/artifact.toml	17 lines
components.json	21 lines
index.html	31 lines
mockupPreviewPlugin.ts	180 lines
package.json	74 lines
src/.generated/mockup-components.ts	5 lines
src/App.tsx	146 lines
src/components/ui/accordion.tsx	55 lines
src/components/ui/alert-dialog.tsx	139 lines
src/components/ui/alert.tsx	59 lines
src/components/ui/aspect-ratio.tsx	5 lines
src/components/ui/avatar.tsx	50 lines
src/components/ui/badge.tsx	37 lines
src/components/ui/breadcrumb.tsx	115 lines
src/components/ui/button-group.tsx	83 lines
src/components/ui/button.tsx	58 lines
src/components/ui/calendar.tsx	213 lines
src/components/ui/card.tsx	76 lines
src/components/ui/carousel.tsx	260 lines
src/components/ui/chart.tsx	365 lines
src/components/ui/checkbox.tsx	28 lines
src/components/ui/collapsible.tsx	11 lines
src/components/ui/command.tsx	153 lines
src/components/ui/context-menu.tsx	198 lines
src/components/ui/dialog.tsx	120 lines
src/components/ui/drawer.tsx	116 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
src/components/ui/dropdown-menu.tsx	201 lines
src/components/ui/empty.tsx	104 lines
src/components/ui/field.tsx	244 lines
src/components/ui/form.tsx	176 lines
src/components/ui/hover-card.tsx	27 lines
src/components/ui/input-group.tsx	165 lines
src/components/ui/input-otp.tsx	69 lines
src/components/ui/input.tsx	22 lines
src/components/ui/item.tsx	193 lines
src/components/ui/kbd.tsx	28 lines
src/components/ui/label.tsx	26 lines
src/components/ui/menubar.tsx	254 lines
src/components/ui/navigation-menu.tsx	128 lines
src/components/ui/pagination.tsx	117 lines
src/components/ui/popover.tsx	31 lines
src/components/ui/progress.tsx	28 lines
src/components/ui/radio-group.tsx	42 lines
src/components/ui/resizable.tsx	45 lines
src/components/ui/scroll-area.tsx	46 lines
src/components/ui/select.tsx	159 lines
src/components/ui/separator.tsx	29 lines
src/components/ui/sheet.tsx	140 lines
src/components/ui/sidebar.tsx	714 lines
src/components/ui/skeleton.tsx	15 lines
src/components/ui/slider.tsx	26 lines
src/components/ui/sonner.tsx	31 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
src/components/ui/spinner.tsx	16 lines
src/components/ui/switch.tsx	27 lines
src/components/ui/table.tsx	120 lines
src/components/ui/tabs.tsx	53 lines
src/components/ui/textarea.tsx	22 lines
src/components/ui/toast.tsx	127 lines
src/components/ui/toaster.tsx	33 lines
src/components/ui/toggle-group.tsx	61 lines
src/components/ui/toggle.tsx	43 lines
src/components/ui/tooltip.tsx	32 lines
src/hooks/use-mobile.tsx	19 lines
src/hooks/use-toast.ts	189 lines
src/index.css	185 lines
src/lib/utils.ts	6 lines
src/main.tsx	5 lines
tsconfig.json	18 lines
vite.config.ts	71 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

B14 / MIGRATION BACKUP / .MIGRATION-BACKUP

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
.gitignore	41 lines
.replit	6 lines
AGENTS.md	5 lines
CLAUDE.md	1 lines
IDEAS.md	39 lines
README.md	36 lines
eslint.config.mjs	18 lines
next.config.ts	29 lines
package-lock.json	8,301 lines
package.json	44 lines
postcss.config.mjs	7 lines
public/file.svg	1 lines
public/globe.svg	1 lines
public/next.svg	1 lines
public/vercel.svg	1 lines
public/window.svg	1 lines
src/app/about/page.tsx	74 lines
src/app/archives/page.tsx	36 lines
src/app/blog/page.tsx	47 lines
src/app/careers/page.tsx	48 lines
src/app/crdt/page.tsx	40 lines
src/app/engine/page.tsx	116 lines
src/app/favicon.ico	25,931 bytes
src/app/features/echo-studio/page.tsx	97 lines
src/app/features/empathy-coach/page.tsx	44 lines
src/app/features/heritage-echoes/page.tsx	43 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

B15 / MIGRATION BACKUP / .MIGRATION-BACKUP · CONTINUED

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
src/app/features/mood-genome/page.tsx	64 lines
src/app/features/page.tsx	46 lines
src/app/features/resoquests/page.tsx	61 lines
src/app/features/safe-circles/page.tsx	46 lines
src/app/founders/page.tsx	106 lines
src/app/globals.css	129 lines
src/app/impact/page.tsx	81 lines
src/app/join/page.tsx	56 lines
src/app/layout.tsx	39 lines
src/app/manifesto/page.tsx	53 lines
src/app/not-found.tsx	39 lines
src/app/page.tsx	195 lines
src/app/privacy/page.tsx	37 lines
src/app/protocol/page.tsx	37 lines
src/app/roadmap/page.tsx	109 lines
src/app/schools/page.tsx	78 lines
src/app/technology/page.tsx	73 lines
src/app/telemetry/page.tsx	42 lines
src/app/template.tsx	8 lines
src/app/terminal/page.tsx	177 lines
src/app/terms/page.tsx	32 lines
src/components/animations/LoadingScreen.tsx	101 lines
src/components/canvas/HeroCanvas.tsx	68 lines
src/components/layout/AppProvider.tsx	17 lines
src/components/layout/LenisProvider.tsx	31 lines
src/core/cqrs/Entity.ts	31 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

B16 / MIGRATION BACKUP / .MIGRATION-BACKUP · CONTINUED

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
src/core/cqrs/EventBus.ts	35 lines
src/core/crdt/VectorClock.ts	121 lines
src/core/memory/GarbageCollector.ts	81 lines
src/domain/entities/ResonanceCircle.ts	60 lines
src/domain/entities/User.ts	48 lines
src/domain/value-objects/EmotionalState.ts	45 lines
src/infrastructure/ai/webgpu/NeuralComputeNode.ts	120 lines
src/infrastructure/audio/AcousticResonator.ts	148 lines
src/infrastructure/crypto/CipherProtocol.ts	63 lines
src/infrastructure/telemetry/AnonymizedMetrics.ts	45 lines
src/infrastructure/webrtc/PeerTopologyMesh.ts	95 lines
src/presentation/canvas/CinematicLoaderCanvas.tsx	100 lines
src/presentation/canvas/MeshTopologyCanvas.tsx	101 lines
src/presentation/canvas/NeuralNetworkCanvas.tsx	93 lines
src/presentation/canvas/SingularityCanvas.tsx	144 lines
src/presentation/canvas/archives/DNAHelixCanvas.tsx	95 lines
src/presentation/canvas/audio/AudioVisualizerCanvas.tsx	60 lines
src/presentation/canvas/fluid/FluidCanvas.tsx	86 lines
src/presentation/canvas/impact/GlobeCanvas.tsx	94 lines
src/presentation/canvas/premium/PrismCanvas.tsx	59 lines
src/presentation/canvas/roadmap/RoadmapCanvas.tsx	76 lines
src/presentation/canvas/vector/VectorFieldCanvas.tsx	110 lines
src/presentation/components/CustomCursor.tsx	65 lines
src/presentation/components/MagneticElement.tsx	51 lines
src/presentation/components/MatrixRain.tsx	73 lines
src/presentation/components/PurgeScreen.tsx	81 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

B17 / MIGRATION BACKUP / .MIGRATION-BACKUP · CONTINUED

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
src/presentation/components/ScrambleText.tsx	43 lines
src/presentation/components/TextReveal.tsx	37 lines
src/presentation/components/animations/CinematicLoader.tsx	153 lines
src/presentation/components/chat/AdvancedChat.tsx	136 lines
src/presentation/components/navigation/GlobalNav.tsx	120 lines
src/presentation/components/transitions/TransitionShutter.tsx	60 lines
src/presentation/components/ui/CinematicText.tsx	41 lines
src/presentation/components/ui/ContextMenu.tsx	55 lines
src/presentation/hooks/useGSAPOrchestrator.ts	37 lines
src/presentation/layouts/AppProvider.tsx	20 lines
src/presentation/layouts/LenisProvider.tsx	31 lines
src/presentation/shaders/glass.frag	32 lines
src/presentation/shaders/glass.vert	12 lines
src/presentation/shaders/neural.frag	11 lines
src/presentation/shaders/neural.vert	132 lines
src/presentation/shaders/singularity.frag	13 lines
src/presentation/shaders/singularity.vert	76 lines
src/presentation/store/empathyStore.ts	29 lines
src/workers/sentiment.worker.ts	40 lines
tsconfig.json	35 lines

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

Complete repository file catalog

PATH (relative to the scope above)	SIZE / SCALE
4b70fb0a802ae4e14bed695f3e576db9_1781772643839.jpg	27,931 bytes
g1h7uuanzez51_1781772614449.jpg	35,195 bytes

Every tracked baseline file is indexed once in this catalog. Third-party node_modules, generated build output, .git internals, and the new walkthrough files are excluded. Line counts are file scale, not a measure of original authorship. Migration and UI scaffolds are listed without implying active runtime use.

C1 / DEPENDENCY REFERENCE · ACTIVE WORKSPACE AND PRESERVED ORIGINAL

Every declared package

PACKAGE	DECLARED VERSION	ROLE / DECLARING SCOPE
@gsap/react	^2.1.2	React/GSAP integration helper · OLD, WEB
@hookform/resolvers	^3.10.0	Form/schema adapters · PREVIEW, WEB
@radix-ui/react-accordion	^1.2.12	Accessible UI primitive: accordion · PREVIEW
@radix-ui/react-accordion	^1.2.4	Accessible UI primitive: accordion · WEB
@radix-ui/react-alert-dialog	^1.1.15	Accessible UI primitive: alert dialog · PREVIEW
@radix-ui/react-alert-dialog	^1.1.7	Accessible UI primitive: alert dialog · WEB
@radix-ui/react-aspect-ratio	^1.1.3	Accessible UI primitive: aspect ratio · WEB
@radix-ui/react-aspect-ratio	^1.1.8	Accessible UI primitive: aspect ratio · PREVIEW
@radix-ui/react-avatar	^1.1.11	Accessible UI primitive: avatar · PREVIEW
@radix-ui/react-avatar	^1.1.4	Accessible UI primitive: avatar · WEB
@radix-ui/react-checkbox	^1.1.5	Accessible UI primitive: checkbox · WEB
@radix-ui/react-checkbox	^1.3.3	Accessible UI primitive: checkbox · PREVIEW
@radix-ui/react-collapsible	^1.1.12	Accessible UI primitive: collapsible · PREVIEW
@radix-ui/react-collapsible	^1.1.4	Accessible UI primitive: collapsible · WEB
@radix-ui/react-context-menu	^2.2.16	Accessible UI primitive: context menu · PREVIEW
@radix-ui/react-context-menu	^2.2.7	Accessible UI primitive: context menu · WEB
@radix-ui/react-dialog	^1.1.15	Accessible UI primitive: dialog · PREVIEW

Versions are manifest declarations/ranges, not a claim about every installed resolution. Catalog references are expanded from pnpm-workspace.yaml. WEB = frontend; API = server; PREVIEW = mockup sandbox; OLD = migration backup. CLIENT / SPEC / VALIDATE / DB / SCRIPTS refer to shared packages. A declared dependency is not evidence of runtime use.

C2 / DEPENDENCY REFERENCE · ACTIVE WORKSPACE AND PRESERVED ORIGINAL

Every declared package

PACKAGE	DECLARED VERSION	ROLE / DECLARING SCOPE
@radix-ui/react-dialog	^1.1.7	Accessible UI primitive: dialog · WEB
@radix-ui/react-dropdown-menu	^2.1.16	Accessible UI primitive: dropdown menu · PREVIEW
@radix-ui/react-dropdown-menu	^2.1.7	Accessible UI primitive: dropdown menu · WEB
@radix-ui/react-hover-card	^1.1.15	Accessible UI primitive: hover card · PREVIEW
@radix-ui/react-hover-card	^1.1.7	Accessible UI primitive: hover card · WEB
@radix-ui/react-label	^2.1.3	Accessible UI primitive: label · WEB
@radix-ui/react-label	^2.1.8	Accessible UI primitive: label · PREVIEW
@radix-ui/react-menubar	^1.1.16	Accessible UI primitive: menubar · PREVIEW
@radix-ui/react-menubar	^1.1.7	Accessible UI primitive: menubar · WEB
@radix-ui/react-navigation-menu	^1.2.14	Accessible UI primitive: navigation menu · PREVIEW
@radix-ui/react-navigation-menu	^1.2.6	Accessible UI primitive: navigation menu · WEB
@radix-ui/react-popover	^1.1.15	Accessible UI primitive: popover · PREVIEW
@radix-ui/react-popover	^1.1.7	Accessible UI primitive: popover · WEB
@radix-ui/react-progress	^1.1.3	Accessible UI primitive: progress · WEB
@radix-ui/react-progress	^1.1.8	Accessible UI primitive: progress · PREVIEW
@radix-ui/react-radio-group	^1.2.4	Accessible UI primitive: radio group · WEB
@radix-ui/react-radio-group	^1.3.8	Accessible UI primitive: radio group · PREVIEW

Versions are manifest declarations/ranges, not a claim about every installed resolution. Catalog references are expanded from pnpm-workspace.yaml. WEB = frontend; API = server; PREVIEW = mockup sandbox; OLD = migration backup. CLIENT / SPEC / VALIDATE / DB / SCRIPTS refer to shared packages. A declared dependency is not evidence of runtime use.

C3 / DEPENDENCY REFERENCE · ACTIVE WORKSPACE AND PRESERVED ORIGINAL

Every declared package

PACKAGE	DECLARED VERSION	ROLE / DECLARING SCOPE
@radix-ui/react-scroll-area	^1.2.10	Accessible UI primitive: scroll area · PREVIEW
@radix-ui/react-scroll-area	^1.2.4	Accessible UI primitive: scroll area · WEB
@radix-ui/react-select	^2.1.7	Accessible UI primitive: select · WEB
@radix-ui/react-select	^2.2.6	Accessible UI primitive: select · PREVIEW
@radix-ui/react-separator	^1.1.3	Accessible UI primitive: separator · WEB
@radix-ui/react-separator	^1.1.8	Accessible UI primitive: separator · PREVIEW
@radix-ui/react-slider	^1.2.4	Accessible UI primitive: slider · WEB
@radix-ui/react-slider	^1.3.6	Accessible UI primitive: slider · PREVIEW
@radix-ui/react-slot	^1.2.0	Accessible UI primitive: slot · WEB
@radix-ui/react-slot	^1.2.4	Accessible UI primitive: slot · PREVIEW
@radix-ui/react-switch	^1.1.4	Accessible UI primitive: switch · WEB
@radix-ui/react-switch	^1.2.6	Accessible UI primitive: switch · PREVIEW
@radix-ui/react-tabs	^1.1.13	Accessible UI primitive: tabs · PREVIEW
@radix-ui/react-tabs	^1.1.4	Accessible UI primitive: tabs · WEB
@radix-ui/react-toast	^1.2.15	Accessible UI primitive: toast · PREVIEW
@radix-ui/react-toast	^1.2.7	Accessible UI primitive: toast · WEB
@radix-ui/react-toggle	^1.1.10	Accessible UI primitive: toggle · PREVIEW

Versions are manifest declarations/ranges, not a claim about every installed resolution. Catalog references are expanded from pnpm-workspace.yaml. WEB = frontend; API = server; PREVIEW = mockup sandbox; OLD = migration backup. CLIENT / SPEC / VALIDATE / DB / SCRIPTS refer to shared packages. A declared dependency is not evidence of runtime use.

C4 / DEPENDENCY REFERENCE · ACTIVE WORKSPACE AND PRESERVED ORIGINAL

Every declared package

PACKAGE	DECLARED VERSION	ROLE / DECLARING SCOPE
@radix-ui/react-toggle	^1.1.3	Accessible UI primitive: toggle · WEB
@radix-ui/react-toggle-group	^1.1.11	Accessible UI primitive: toggle group · PREVIEW
@radix-ui/react-toggle-group	^1.1.3	Accessible UI primitive: toggle group · WEB
@radix-ui/react-tooltip	^1.2.0	Accessible UI primitive: tooltip · WEB
@radix-ui/react-tooltip	^1.2.8	Accessible UI primitive: tooltip · PREVIEW
@react-three/drei	^10.7.7	React Three helper components · OLD, WEB
@react-three/fiber	^9.6.1	React renderer for Three.js · OLD, WEB
@react-three/postprocessing	^3.0.4	React postprocessing wrappers · OLD, WEB
@replit/vite-plugin-cartographer	^0.5.21 [catalog]	Replit development/preview tooling · PREVIEW, WEB
@replit/vite-plugin-dev-banner	^0.1.1 [catalog]	Replit development/preview tooling · WEB
@replit/vite-plugin-runtime-error-modal	^0.0.6 [catalog]	Replit development/preview tooling · PREVIEW, WEB
@tailwindcss/postcss	^4	Original Next/Tailwind integration · OLD
@tailwindcss/typography	^0.5.15	Prose styling extension · WEB
@tailwindcss/vite	^4.1.14 [catalog]	Tailwind build integration · PREVIEW, WEB
@tanstack/react-query	^5.90.21 [catalog]	Async query client/provider · WEB, CLIENT
@types/animejs	^3.1.13	Original Anime.js type support · OLD
@types/cookie-parser	^1.4.10	Type declarations for cookie-parser · API

Versions are manifest declarations/ranges, not a claim about every installed resolution. Catalog references are expanded from pnpm-workspace.yaml. WEB = frontend; API = server; PREVIEW = mockup sandbox; OLD = migration backup. CLIENT / SPEC / VALIDATE / DB / SCRIPTS refer to shared packages. A declared dependency is not evidence of runtime use.

C5 / DEPENDENCY REFERENCE · ACTIVE WORKSPACE AND PRESERVED ORIGINAL

Every declared package

PACKAGE	DECLARED VERSION	ROLE / DECLARING SCOPE
@types/cors	^2.8.19	Type declarations for cors · API
@types/express	^5.0.6	Type declarations for express · API
@types/node	^20	Node platform type declarations · OLD
@types/node	^25.3.3 [catalog]	Node platform type declarations · API, PREVIEW, WEB, DB, SCRIPTS
@types/pg	^8.20.0	Type declarations for pg · DB
@types/react	^19	React type declarations · OLD
@types/react	^19.2.0 [catalog]	React type declarations · PREVIEW, WEB
@types/react-dom	^19	React DOM type declarations · OLD
@types/react-dom	^19.2.0 [catalog]	React DOM type declarations · PREVIEW, WEB
@types/three	^0.184.1	Original Three declarations · OLD
@vitejs/plugin-react	^5.0.4 [catalog]	Supporting package; see declaring manifest · PREVIEW, WEB
@webgpu/types	^0.1.70	Original browser GPU declarations · OLD
@workspace/api-client-react	workspace:*	Internal workspace package link · WEB
@workspace/api-zod	workspace:*	Internal workspace package link · API
@workspace/db	workspace:*	Internal workspace package link · API
animejs	^3.2.2	Animation library; v4 active/v3 backup · OLD
animejs	^4.4.1	Animation library; v4 active/v3 backup · WEB

Versions are manifest declarations/ranges, not a claim about every installed resolution. Catalog references are expanded from pnpm-workspace.yaml. WEB = frontend; API = server; PREVIEW = mockup sandbox; OLD = migration backup. CLIENT / SPEC / VALIDATE / DB / SCRIPTS refer to shared packages. A declared dependency is not evidence of runtime use.

C6 / DEPENDENCY REFERENCE · ACTIVE WORKSPACE AND PRESERVED ORIGINAL

Every declared package

PACKAGE	DECLARED VERSION	ROLE / DECLARING SCOPE
chokidar	^4.0.3	Preview file watching · PREVIEW
class-variance-authority	^0.7.1 [catalog]	Variant-based class composition · PREVIEW, WEB
clsx	^2.1.1	Conditional class strings · OLD
clsx	^2.1.1 [catalog]	Conditional class strings · PREVIEW, WEB
cmdk	^1.1.1	Command-menu UI primitive · PREVIEW, WEB
cookie-parser	^1.4.7	Cookie parsing scaffold · API
cors	^2.8.6	Cross-origin middleware · API
date-fns	^3.6.0	Date utilities · PREVIEW, WEB
drizzle-kit	^0.31.10	Schema tooling · DB
drizzle-orm	^0.45.2 [catalog]	Database query/model layer · API, DB
drizzle-zod	^0.8.3	DB-to-validation helpers · DB
embla-carousel-react	^8.6.0	Carousel behavior · PREVIEW, WEB
esbuild	0.27.3	API bundler / build tool · API
esbuild-plugin-pino	^2.3.3	Pino bundling integration · API
eslint	^9	Original lint tooling · OLD
eslint-config-next	16.2.9	Original Next lint rules · OLD
express	^5.2.1	HTTP service framework · API

Versions are manifest declarations/ranges, not a claim about every installed resolution. Catalog references are expanded from pnpm-workspace.yaml. WEB = frontend; API = server; PREVIEW = mockup sandbox; OLD = migration backup. CLIENT / SPEC / VALIDATE / DB / SCRIPTS refer to shared packages. A declared dependency is not evidence of runtime use.

C7 / DEPENDENCY REFERENCE · ACTIVE WORKSPACE AND PRESERVED ORIGINAL

Every declared package

PACKAGE	DECLARED VERSION	ROLE / DECLARING SCOPE
fast-glob	^3.3.3	Preview component discovery · PREVIEW
framer-motion	^12.40.0	Declarative UI animation · OLD
framer-motion	^12.23.24 [catalog]	Declarative UI animation · PREVIEW, WEB
gsap	^3.15.0	Imperative animation and scroll tooling · OLD, WEB
input-otp	^1.4.2	OTP input primitive · PREVIEW, WEB
lenis	^1.3.23	Smooth scroll engine · OLD, WEB
lucide-react	^1.17.0	Interface icons · OLD
lucide-react	^0.545.0 [catalog]	Interface icons · PREVIEW, WEB
next	16.2.9	Preserved original framework, not active app · OLD
next-themes	^0.4.6	Theme helper; not Next runtime proof · PREVIEW, WEB
orval	^8.9.1	OpenAPI code generation · SPEC
pg	^8.20.0	PostgreSQL driver · DB
pino	^9.14.0	Structured logger · API
pino-http	^10.5.0	HTTP logging middleware · API
pino-pretty	^13.1.3	Development log formatting · API
prettier	^3.8.3	Code formatting · ROOT
raw-loader	^4.0.2	Original shader/raw import loader · OLD

Versions are manifest declarations/ranges, not a claim about every installed resolution. Catalog references are expanded from pnpm-workspace.yaml. WEB = frontend; API = server; PREVIEW = mockup sandbox; OLD = migration backup. CLIENT / SPEC / VALIDATE / DB / SCRIPTS refer to shared packages. A declared dependency is not evidence of runtime use.

C8 / DEPENDENCY REFERENCE · ACTIVE WORKSPACE AND PRESERVED ORIGINAL

Every declared package

PACKAGE	DECLARED VERSION	ROLE / DECLARING SCOPE
react	19.2.4	UI component and state runtime · OLD
react	19.1.0 [catalog]	UI component and state runtime · PREVIEW, WEB
react-day-picker	^9.11.1	Calendar selection UI · WEB
react-day-picker	^9.14.0	Calendar selection UI · PREVIEW
react-dom	19.2.4	React browser rendering · OLD
react-dom	19.1.0 [catalog]	React browser rendering · PREVIEW, WEB
react-hook-form	^7.55.0	Form management library · WEB
react-hook-form	^7.75.0	Form management library · PREVIEW
react-icons	^5.4.0	Additional icon library · WEB
react-resizable-panels	^2.1.7	Resizable layout panels · WEB
react-resizable-panels	^2.1.9	Resizable layout panels · PREVIEW
recharts	^2.15.2	Chart UI library · WEB
recharts	^2.15.4	Chart UI library · PREVIEW
sonner	^2.0.7	Toast notifications · PREVIEW, WEB
tailwind-merge	^3.6.0	Tailwind class conflict resolution · OLD
tailwind-merge	^3.3.1 [catalog]	Tailwind class conflict resolution · PREVIEW, WEB
tailwindcss	^4	Utility-first styling · OLD

Versions are manifest declarations/ranges, not a claim about every installed resolution. Catalog references are expanded from pnpm-workspace.yaml. WEB = frontend; API = server; PREVIEW = mockup sandbox; OLD = migration backup. CLIENT / SPEC / VALIDATE / DB / SCRIPTS refer to shared packages. A declared dependency is not evidence of runtime use.

C9 / DEPENDENCY REFERENCE · ACTIVE WORKSPACE AND PRESERVED ORIGINAL

Every declared package

PACKAGE	DECLARED VERSION	ROLE / DECLARING SCOPE
tailwindcss	^4.1.14 [catalog]	Utility-first styling · PREVIEW, WEB
tailwindcss-animate	^1.0.7	Supporting package; see declaring manifest · PREVIEW
thread-stream	3.1.0	Logging worker transport · API
three	^0.184.0	3D rendering objects and materials · OLD, WEB
three-stdlib	^2.36.1	Three helper implementations · OLD, WEB
tsx	^4.21.0 [catalog]	TypeScript script runner · SCRIPTS
tw-animate-css	^1.4.0	Animation utility styles · PREVIEW, WEB
typescript	^5	Static type checking · OLD
typescript	~5.9.3	Static type checking · ROOT
vaul	^1.1.2	Drawer primitive · PREVIEW, WEB
vite	^7.3.2 [catalog]	Frontend dev server and bundler · PREVIEW, WEB
wouter	^3.3.5	Client-side routing · WEB
zod	^3.25.76 [catalog]	Validation and schema types · PREVIEW, WEB, VALIDATE, DB
zustand	^5.0.14	Lightweight local state store · OLD, WEB

Versions are manifest declarations/ranges, not a claim about every installed resolution. Catalog references are expanded from pnpm-workspace.yaml. WEB = frontend; API = server; PREVIEW = mockup sandbox; OLD = migration backup. CLIENT / SPEC / VALIDATE / DB / SCRIPTS refer to shared packages. A declared dependency is not evidence of runtime use.

D1 / USE THE DOCUMENT WITH CONFIDENCE

Sources, reproducibility, and credits

Source of truth

This document is grounded in the local checkout of `UCHIHA-MADARA-ANUJ/Resonate-Portfolio` at baseline `14fe27732222a28335c2cb7d45ea5fdf3bf95f8e`. The repository has shallow history here; no detailed individual contribution timeline is inferred. The source catalog covers 370 tracked baseline files. Lockfiles were used for dependency evidence, not reproduced as pages of package-manager internals.

Evidence bundle

`walkthrough/evidence/frontend-build.txt` and `typecheck.txt` preserve the command output described in Chapter 28. `source-manifest.json` records every indexed path, file size, and SHA-256 digest so the source snapshot can be compared later. The manifest excludes walkthrough deliverables and installed/build artifacts. No secret environment file or Git credential data is included.

Rebuild the PDF

The readable manuscript is `walkthrough/walkthrough.md`. `walkthrough/build_pdf.py` parses its chapter markers, adds the full source/dependency catalog from tracked files, and draws the diagrams and page layouts. It uses local DejaVu fonts and ReportLab; it does not fetch web content or images. Install the documented Python dependency in an isolated environment and run the script from the repository checkout. Rebuilding after source edits will appropriately update the catalog.

```
python3 -m venv .venv-docs
.venv-docs/bin/pip install -r walkthrough/requirements.txt
.venv-docs/bin/python walkthrough/build_pdf.py
```

Attribution and limits

The active founder profiles identify Anuj Phulera with development/AI/backend and Aarav Choudhary with vision/marketing/ideas. Older planning notes differ; confirm personal credits before public use. The two attached founder images are stylized anime assets. This PDF does not reuse them, and no ownership or license claim is made for those images. The root manifest declares MIT but a standalone license file is not present in the indexed baseline.

This is a technical walkthrough, not medical advice, a security certification, a verified social-impact report, or a legal-compliance assessment. Recommendations and first-person scripts are clearly editorial. Diagrams are explanatory vector illustrations—not screenshots or representations of a deployed service.

Final note: the strongest version of the project story combines ambition with precision. Show what you built, explain how it works, acknowledge what is simulated, and make the next step testable.